Tema 2.3: Desarrollo de aplicaciones para móviles – Navegación y Comunicaciones

Laura Sacristán Matesanz / David López Cuenca

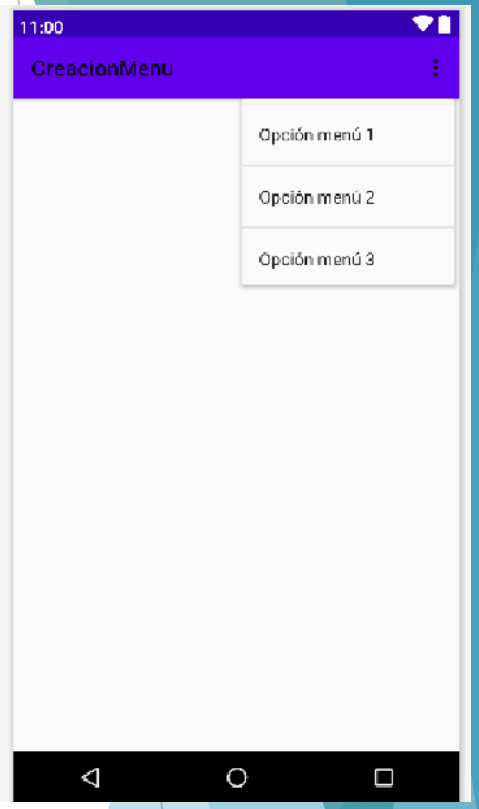
Objetivos del tema

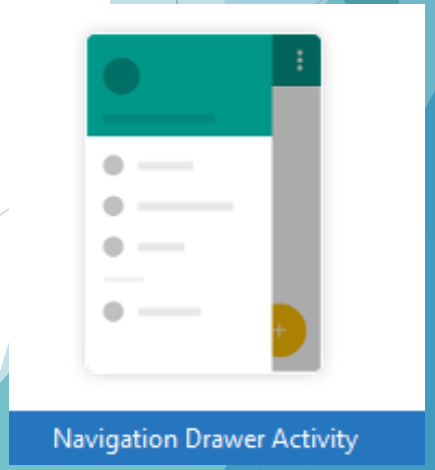
- Navegación de la aplicación.
- Configurar de manera correcta permisos, servicios y conexiones a Internet.
- Configurar las conexiones Bluetooth.
- Realizar conexiones HTTP/HTTPS desde nuestra aplicación.

Navegación (Navigation)

Navegación

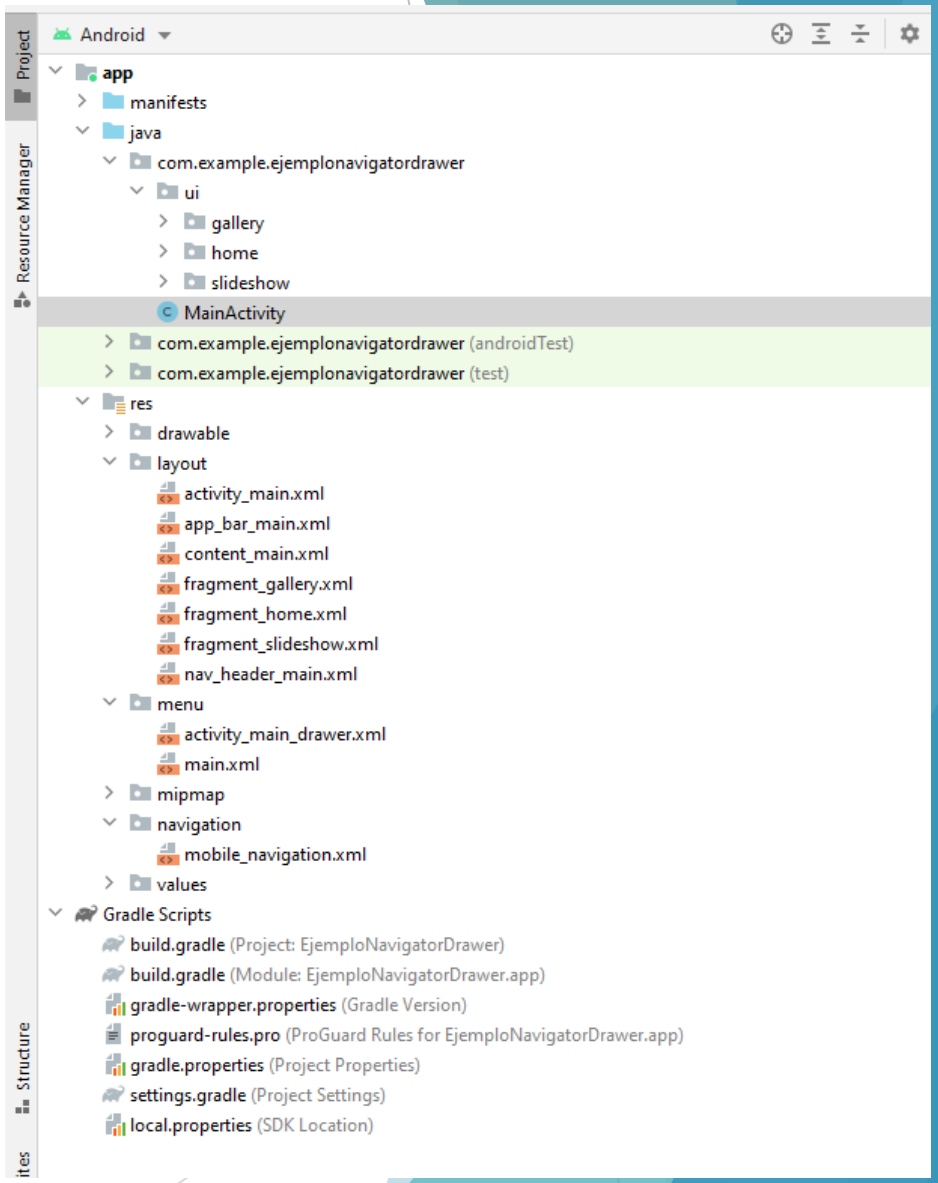
- En el tema anterior vimos como se configuraba un menú de opciones en el que generábamos diferentes elementos que queríamos que aparecieran.
- En este caso, lo que vamos a generar es un NavigatorDrawer que es un menú lateral que es muy utilizado en las aplicaciones para móviles.
- Vamos a hacer un ejemplo para generar este tipo de menú.
- Crearemos un proyecto nuevo que le vamos a llamar EjemploNavigatorDrawer
- En este caso la plantilla que vamos a elegir será una plantilla preparada especialmente para este tipo de menús que es Navigator Drawer Activity.





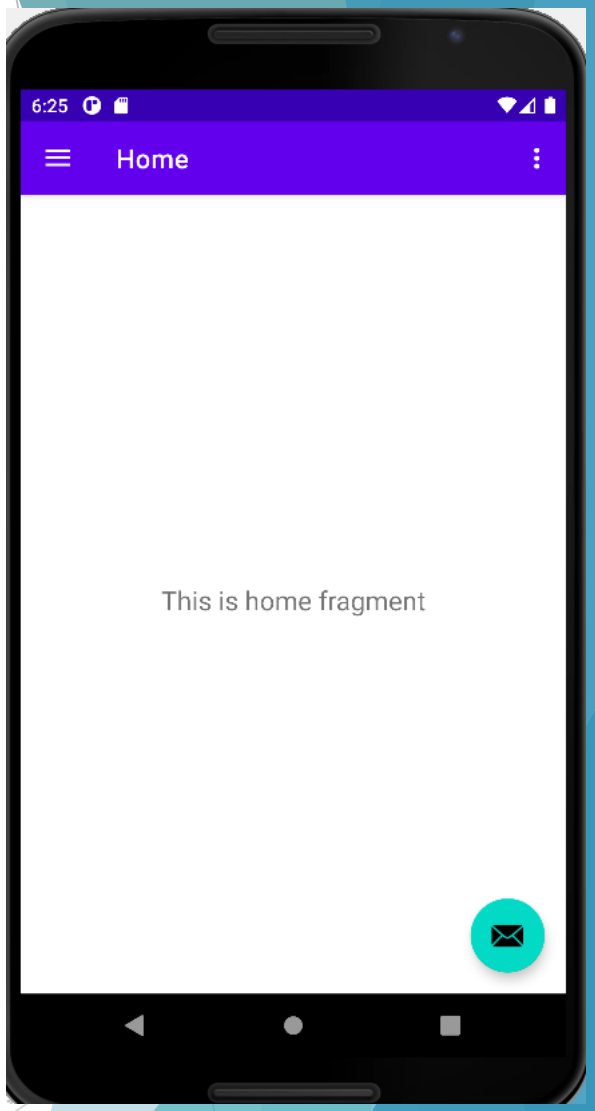
Navegación

- Esta plantilla carga por defecto un fragmento inicial y tres opciones en el menú, que son Home, Gallery y Slideshow, que también tienen sus fragmentos asociados.
- Si miramos en la carpeta Layout tendremos diseñados todos esos elementos que aparecen ya en nuestra plantilla y también tenemos en la carpeta java, unas carpetas creadas para cada una de nuestras opciones del menú.
- Para poder utilizar las opciones de navegación en el archivo build.gradle tendremos implementadas las dependencias de navegación.

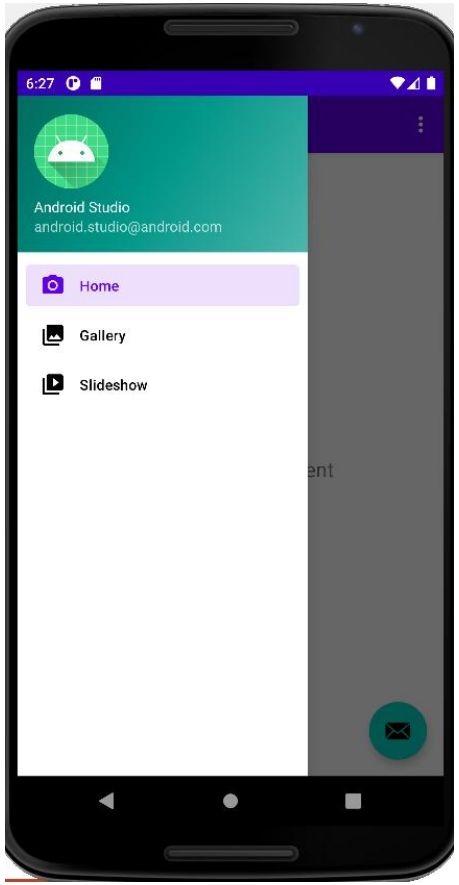


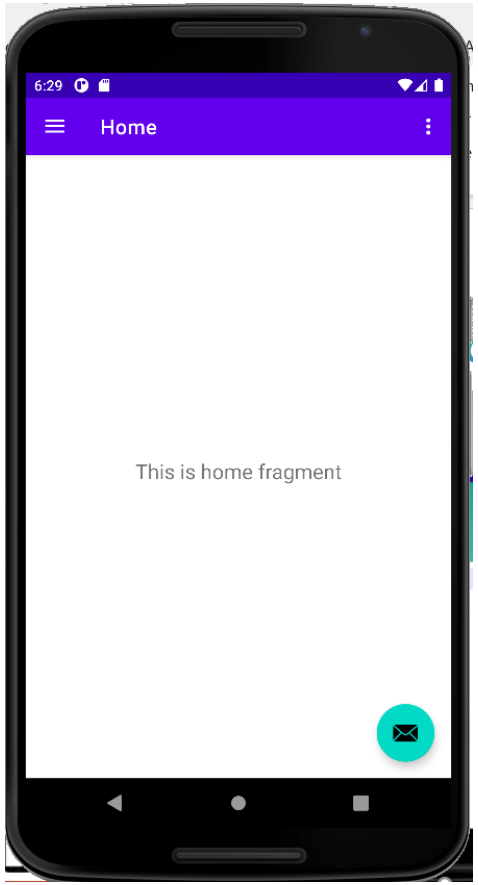
Navegación

- Si ejecutamos la aplicación aparece una pantalla de inicio que es el fragment home.
- Además aparecen dos menús: el drawer que es un menú con las tres opciones: Home, Gallery y Slideshow.
- Cuando pulsamos las opciones del menú carga en el contenedor los fragmentos que tienen asociado.
- Y hay otro menú (a la derecha indicados con los tres puntos) que tiene la opción de setting. Esta opción no está programada.
- También aparece el botón con el sobre para el correo que también está sin programar.



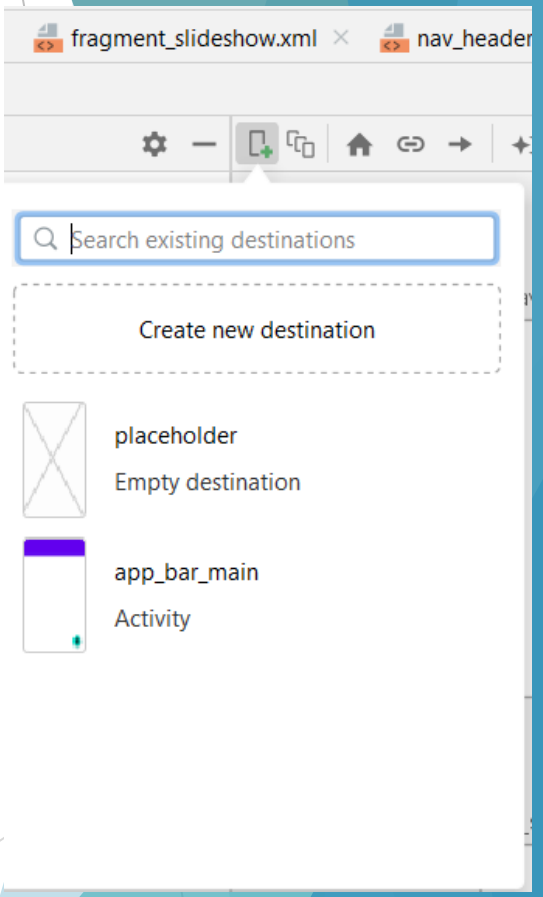
Navegación


6:27
Android Studio
android.studio@android.com
Home
Gallery
Slideshow


6:29
Home
This is home fragment

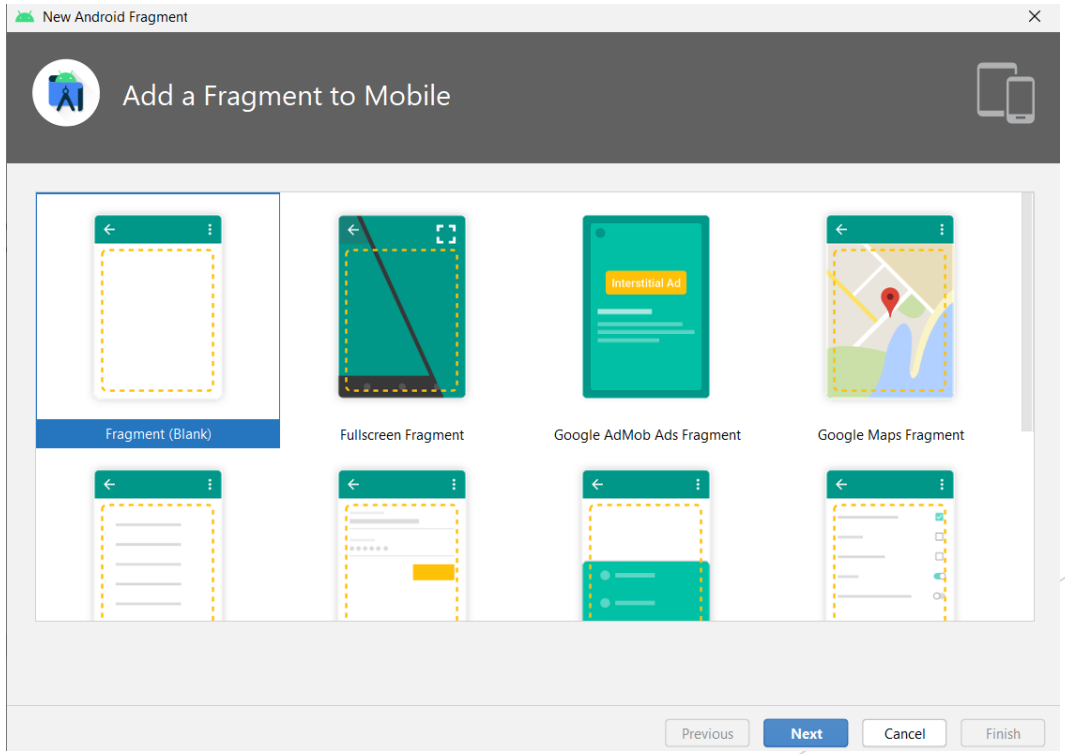
Navegación

- Si revisamos todos los archivos que ha generado el programa, tenemos todos los XML que hacen el diseño de la aplicación base de la plantilla que hemos elegido.
- Para poder añadir más elementos al menú de nuestro programa vamos a ir a al archivo mobile_navigation.xml y vamos a añadir un nuevo destino.
- El contenido de este archivo es los tres diferentes fragmentos que de momento tenemos creados.
- Al pulsar la opción de crear un nuevo destino damos a la opción Create new destination.
- Si tuviéramos algún fragmento ya creado y no incluido en el flujo de navegación, podríamos seleccionarlo de la lista que aparece. No es nuestro caso ya que no tenemos creado ningún fragment sin asignar.



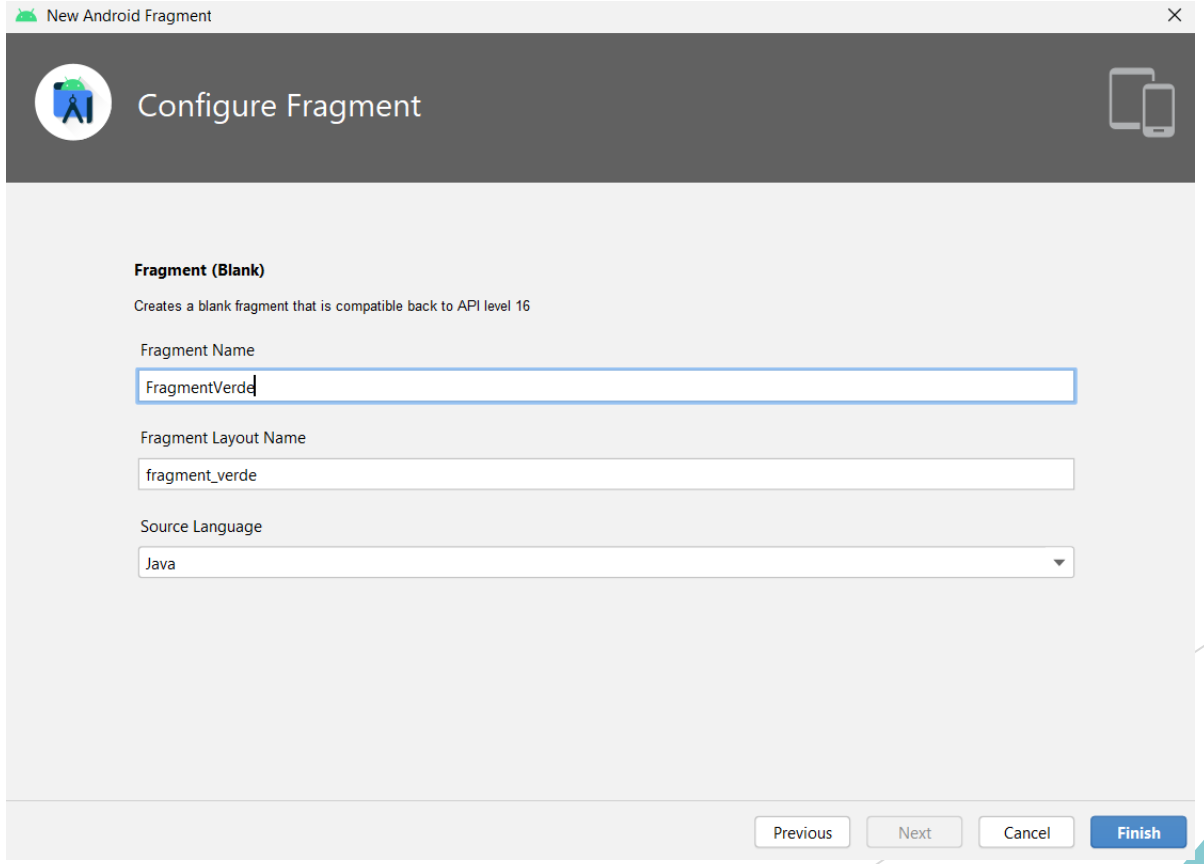
Navegación

- Al pulsar la opción que hemos indicado anteriormente, nos aparece un asistente para añadir un fragmento al programa. Vamos a elegir la plantilla de blankfragment.



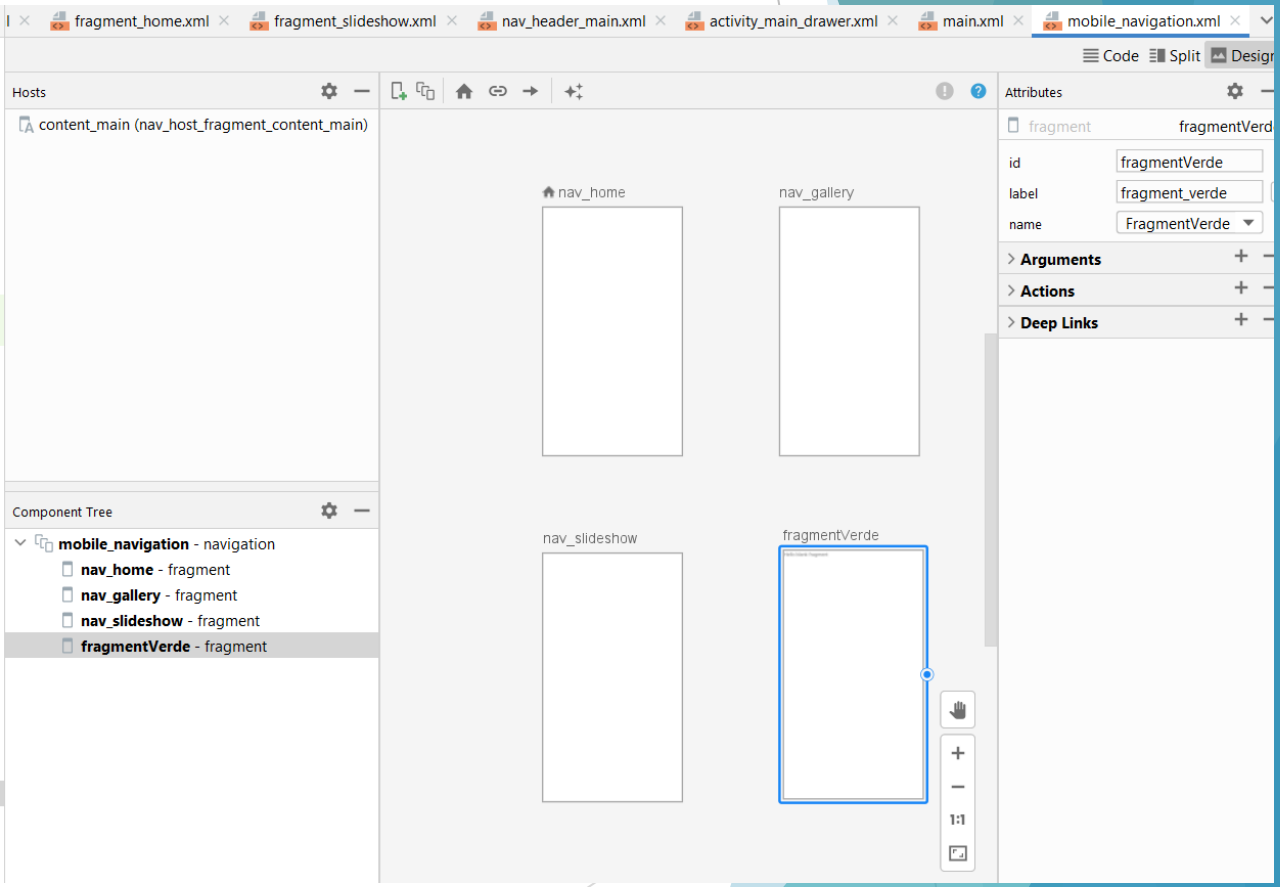
Navegación

- Lo siguiente que nos pide es el nombre del fragmento. Le vamos a llamar FragmentVerde.



Navegación

- En el archivo mobile_navigation.xml aparece el nuevo fragmento.
- Si se necesitara relacionar algún fragmento con otro se podría realizar desde este mismo archivo.
- En este caso no lo vamos a relacionar con ninguno de los existentes porque lo que únicamente queremos que al pulsar una opción del menú se abra el fragmento.

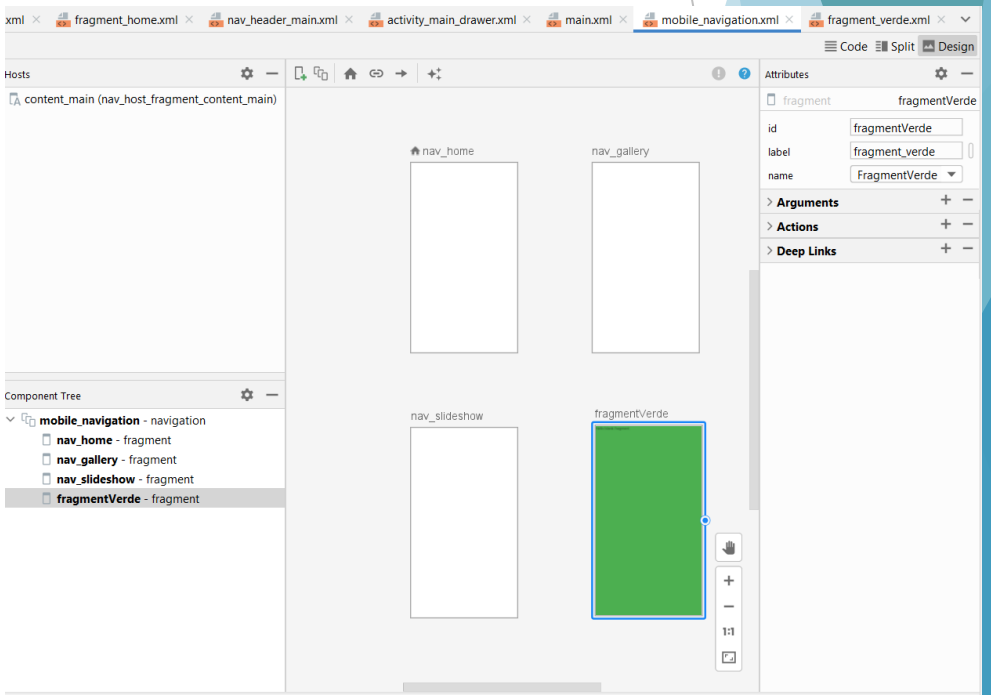


Navegación

- Si hacemos doble click sobre cualquiera de los fragment se nos abre su archivo xml.
- Desde el archivo xml haremos los cambios que necesitemos hacer. En este caso, como le hemos llamado FragmentVerde vamos a ponerle ese color de fondo.
- Cuando hagamos el cambio, si volvemos al

  mobile_navigation veremos que nuestro fragment

  ya tiene aplicados los cambios.



Navegación

- El siguiente paso es ir al archivo activity_main_drawer, que es el archivo que tiene las opciones del menú drawer, y añadimos una nueva opción para que al pulsarla nos muestre el FragmentVerde. Revisar que el id del item tiene el nombre del fragment.
- Copiamos uno de los item que ya están creados y lo modificaremos para tener nuestra nueva opción. Cambiaremos los valores tal y como muestra la imagen.
- Como vemos hay un error en el icono. Eso sucede porque no tenemos creado en la carpeta drawable nuestro xml con la imagen que queremos que nos muestre esta opción del menú. Si pulsamos en la ayuda nos dará la opción de crearla automáticamente.
- Nos salen varias opciones y elegimos la primear

fragment_home.xml | activity_main_drawer.xml | side_nav_bar.xml | mai

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:showIn="navigation_view">

    <group android:checkableBehavior="single">
        <item
            android:id="@+id/nav_home"
            android:icon="@drawable/ic_menu_camera"
            android:title="Home" />
        <item
            android:id="@+id/nav_gallery"
            android:icon="@drawable/ic_menu_gallery"
            android:title="Gallery" />
        <item
            android:id="@+id/nav_slideshow"
            android:icon="@drawable/ic_menu_slideshow"
            android:title="Slideshow" />
        <item
            android:id="@+id/fragmentVerde"
            android:icon="@drawable/ic_menu_share"
            android:title="Fragmento Verde" />
    </group>
</menu>
```

```
android:icon="@drawable/ic_m
```

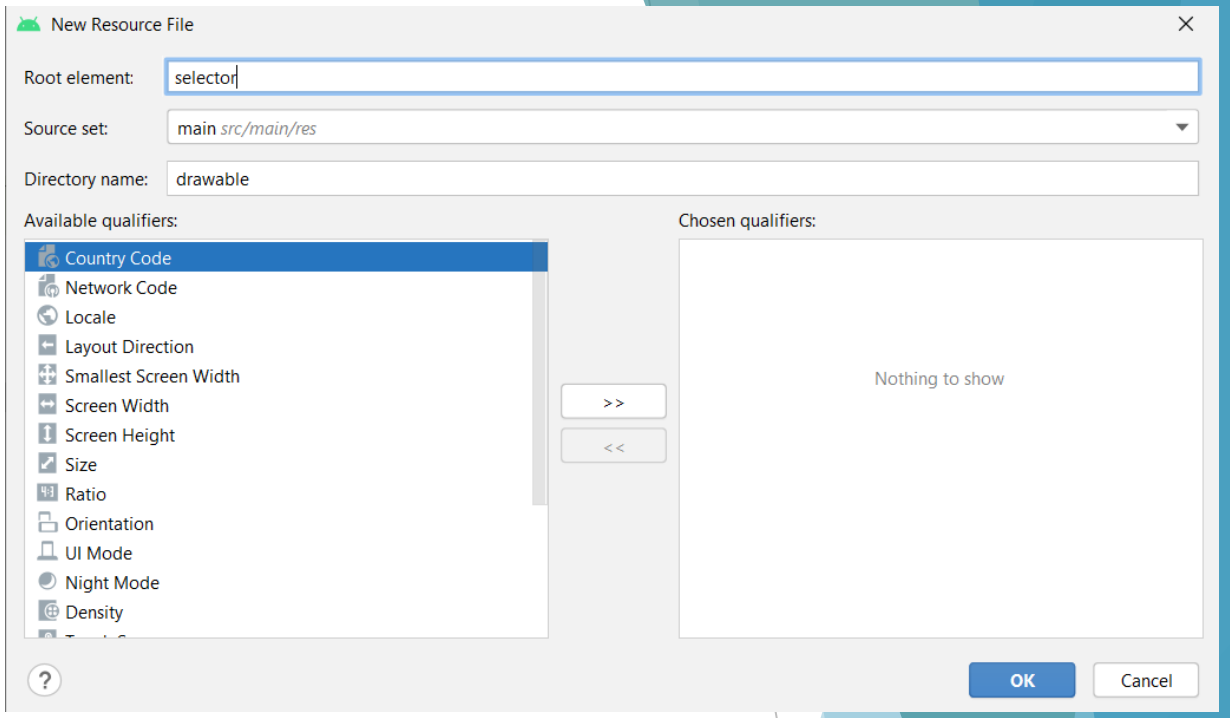
Create drawable resource file 'ic_menu_share.xml'

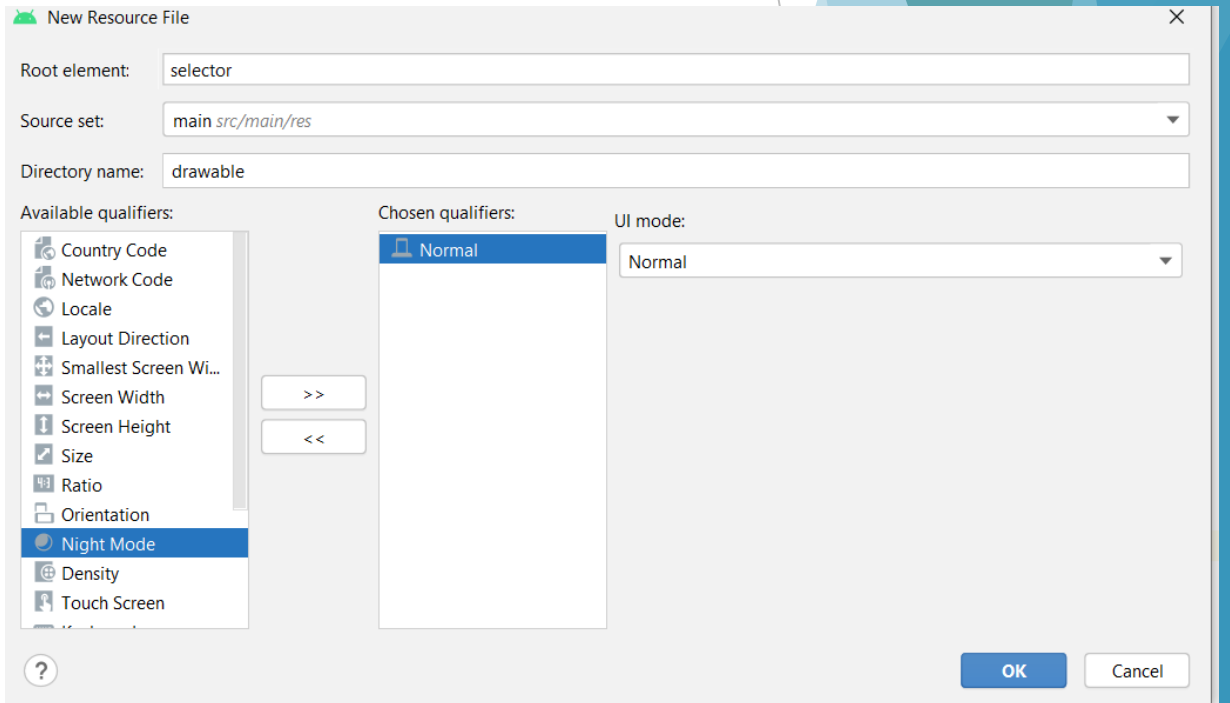
Create drawable value resource 'ic_menu_share'

Press Ctrl+Mayús+I to open preview

Navegación

- Al pulsar la ayuda nos sale la siguiente pantalla.
- Elegimos una de las opciones como por ejemplo UI Mode.
- Nos parecerá la segunda pantalla y le damos a aceptar.
- Nos creará dentro de la carpeta drawable un documento xml que tendrá esta estructura.





Navegación

File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help EjemploNavigatorDrawer - ic_menu_share.xml [EjemploNavigatorDrawer.app]

EjemploNavigatorDrawer > app > src > main > res > drawable > ic_menu_share.xml

app | Nexus 6 API 30

Android

- ui
  - gallery
  - home
  - slideshow
  - FragmentVerde
  - MainActivity
- com.example.ejemplonavigatordrawer (androidTest)
- com.example.ejemplonavigatordrawer (test)
- java (generated)
- res
  - drawable
    - ic_launcher_background.xml
    - ic_launcher_foreground.xml (v24)
    - ic_menu_camera.xml
    - ic_menu_gallery.xml
    - ic_menu_share.xml
    - ic_menu_slideshow.xml
    - side_nav_bar.xml
  - layout
    - activity_main.xml
    - app_bar_main.xml
    - content_main.xml
    - fragment_gallery.xml
    - fragment_home.xml
    - fragment_slideshow.xml
    - fragment_verde.xml
    - nav_header_main.xml
  - menu
    - activity_main_drawer.xml
    - main.xml
  - mipmap
    - ic_launcher (6)
    - ic_launcher_round (6)
  - navigation
    - mobile_navigation.xml

allery.xml | fragment_home.xml | activity_main_drawer.xml | ic_menu_share.xml | ic_menu_gallery.xml | ic_menu_slideshow.xml | side_nav_b

Code | Split | Design

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <selector xmlns:android="http://schemas.android.com/apk/res/android">
3
4 </selector>
```

State Selector

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Run | Event Log | Layout Inspector

Launch succeeded (2 minutes ago)

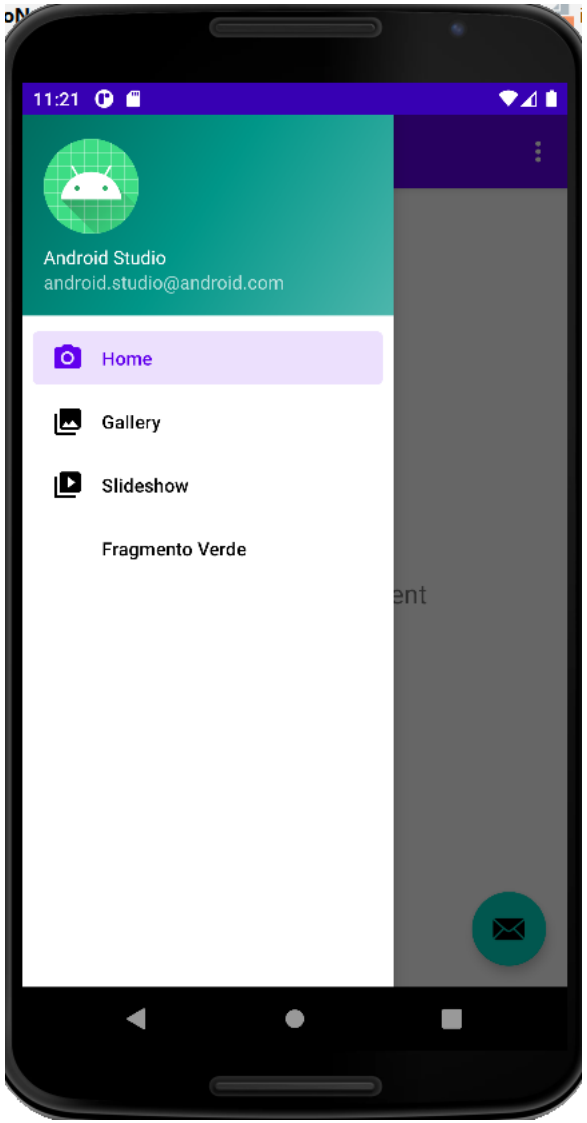
1:1 CRLF UTF-8 4 spaces

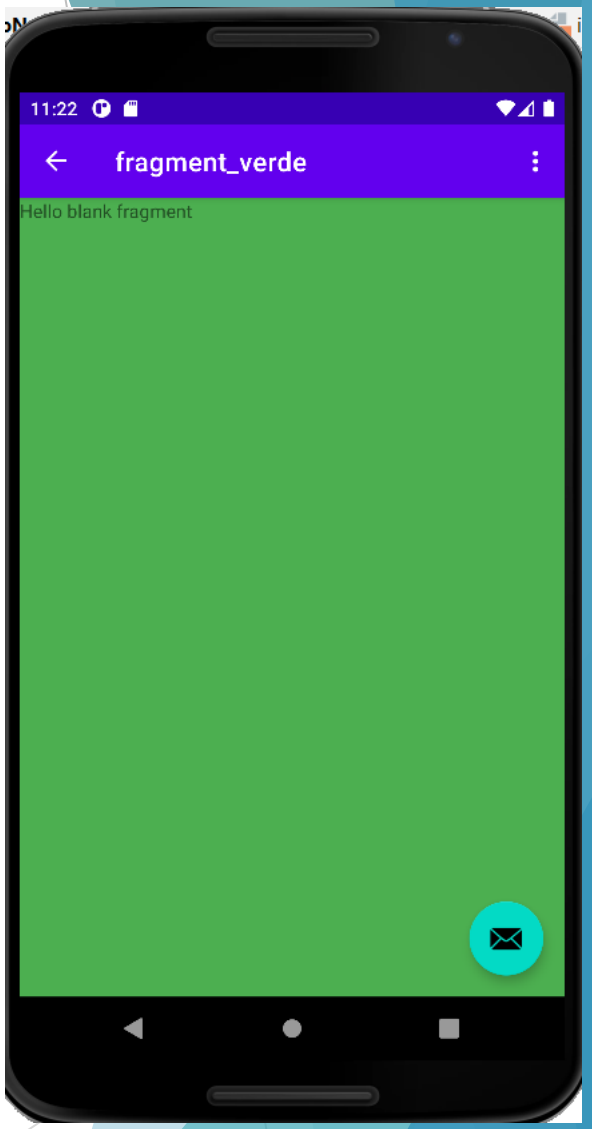
Escribe aquí para buscar

2°C Mayorm. nublado 0:19 23/01/2022

Navegación

- Si ejecutamos la aplicación y accedemos al menú veremos que nos muestra nuestra opción de menú pero no tiene imagen como las otras opciones de menú pero si lo pulsamos funcionará correctamente y mostrará el fragment verde que hemos creado.
- Pero aún nos queda una cosa por mejorar. En los otros fragment, la fecha de arriba a la izquierda que se muestra en el fragment verde no aparece, si no que aparece el acceso al menú.



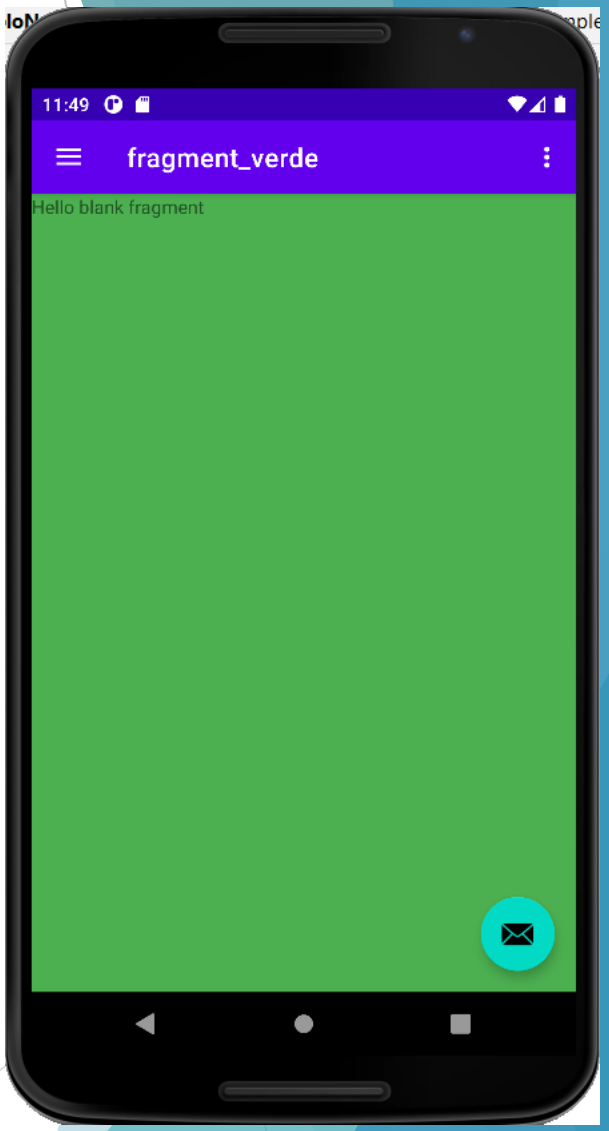


Navegación

- Para poder solucionar esto, debemos acceder al archivo MainActivity y añadir nuestro fragment en la línea

```
mAppBarConfiguration = new AppBarConfiguration.Builder(
        R.id.nav_home, R.id.nav_gallery, R.id.nav_slideshow, R.id.fragmentVerde)
        .setOpenableLayout(drawer)
        .build();
```

- De esta manera conseguimos que en lugar de la flecha aparezca el acceso al menú.



Navegación

- Podemos personalizar tanto la parte de la cabecera del menú, como la parte de las opciones del menú en sí.
- Para poder modificar la parte de la cabecera, deberemos acceder al xml que se llama nav_header_main.xml.
- Dentro de ese fichero está la cabecera que se ha generado por defecto. Nosotros podríamos modificar la imagen y el texto que aparece.
- Por ejemplo, podríamos poner un icono de avatar y como texto nuestro nombre y nuestro correo.
- Si queremos que sea estático, es decir la misma apariencia siempre, podemos modificar directamente el fichero.
- Si queremos que se haga en tiempo de ejecución deberemos hacerlo en el MainActivity.java.

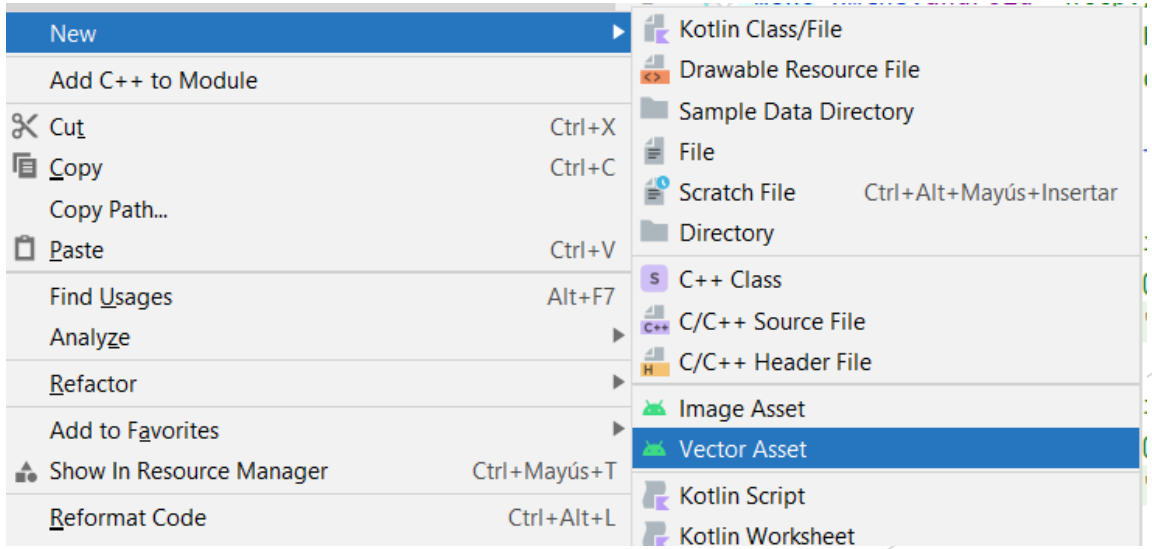
Navegación

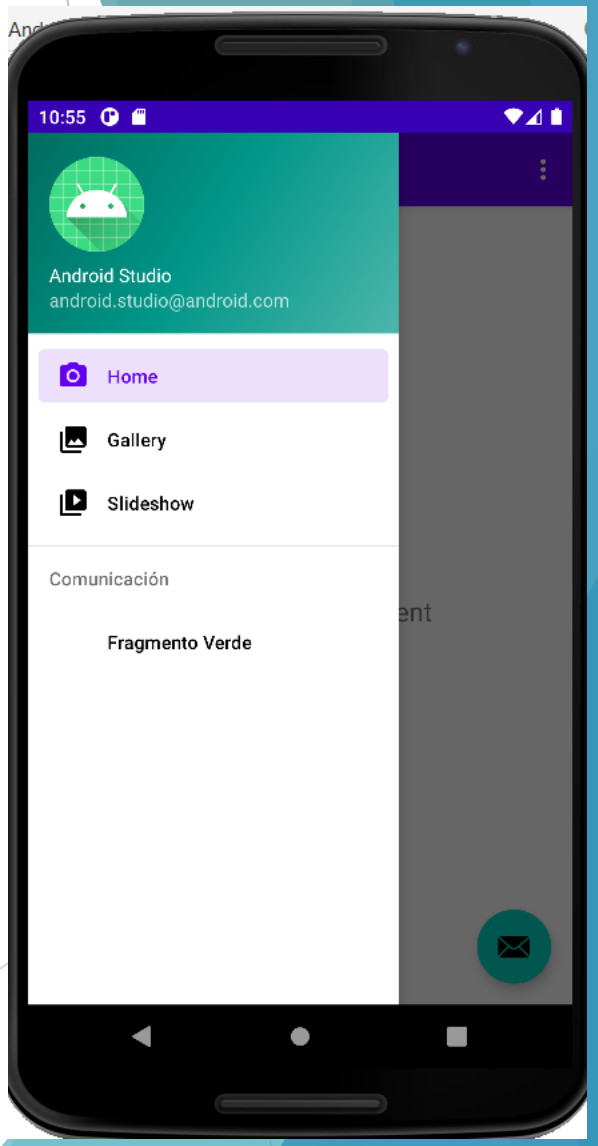
- En el caso de las opciones del menú tendremos que modificar el archivo activity_main_drawer.xml
- Por ejemplo podríamos poner diferentes iconos de los que aparecen y cambiar el nombre de las opciones del menú.
- También podemos dividirlo en bloques diferentes, dependiendo por ejemplo de la funcionalidad o de las diferentes operaciones que pueda realizar nuestra aplicación.
- Si vemos el código del archivo xml, veremos que tiene creado un solo grupo de elementos que sería el que nos genera por defecto que es el SINGLE.
- Si queremos hacer otro bloque tendremos que poner

```
<item android:title="Comunicación">
    <menu>
        <item
            android:id="@+id/fragmentVerde"
            android:icon="@drawable/ic_menu_share"
            android:title="Fragmento Verde" />
    </menu>
</item>
```

Navegación

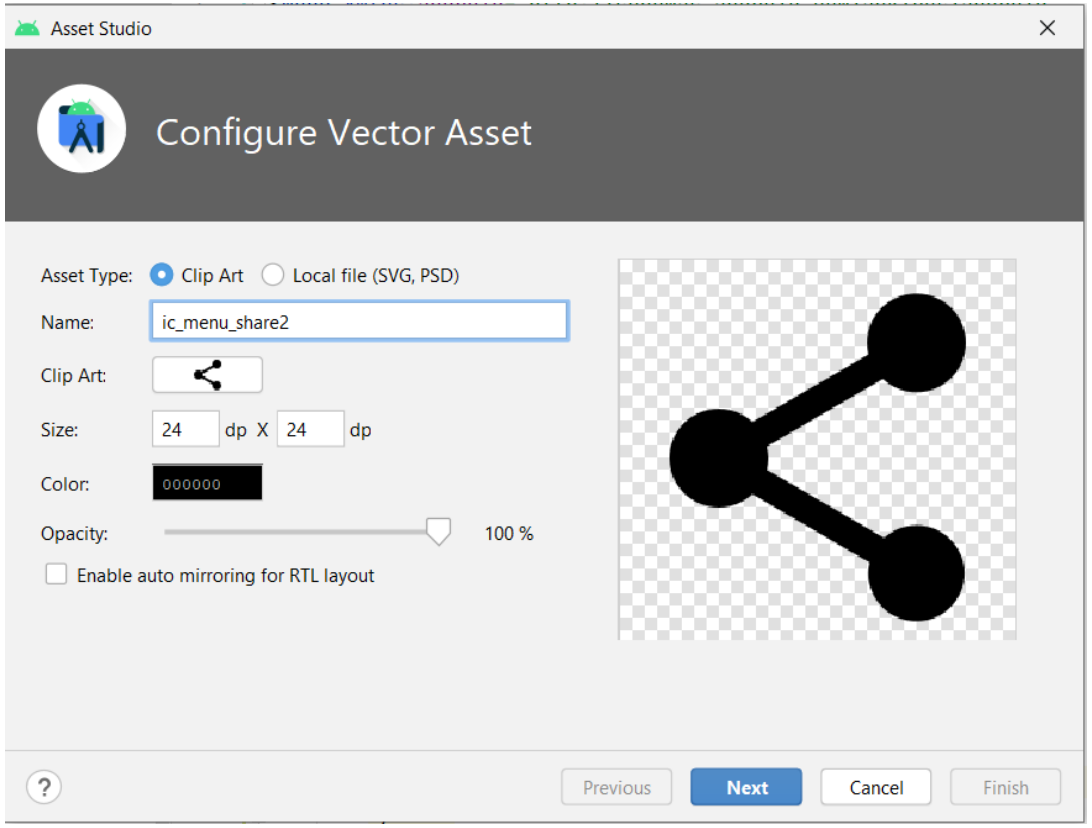
- El resultado sería que nuestra opción Fragmento Verde, ya no está dentro de las opciones iniciales, si no que se ha incluido en otra parte del menú que se llama Comunicación.
- También podemos cambiar los iconos o añadirlos a las opciones que no tienen.
- Para ello vamos a añadir un icono en la carpeta drawable. Damos a new y a la opción Vector Asset

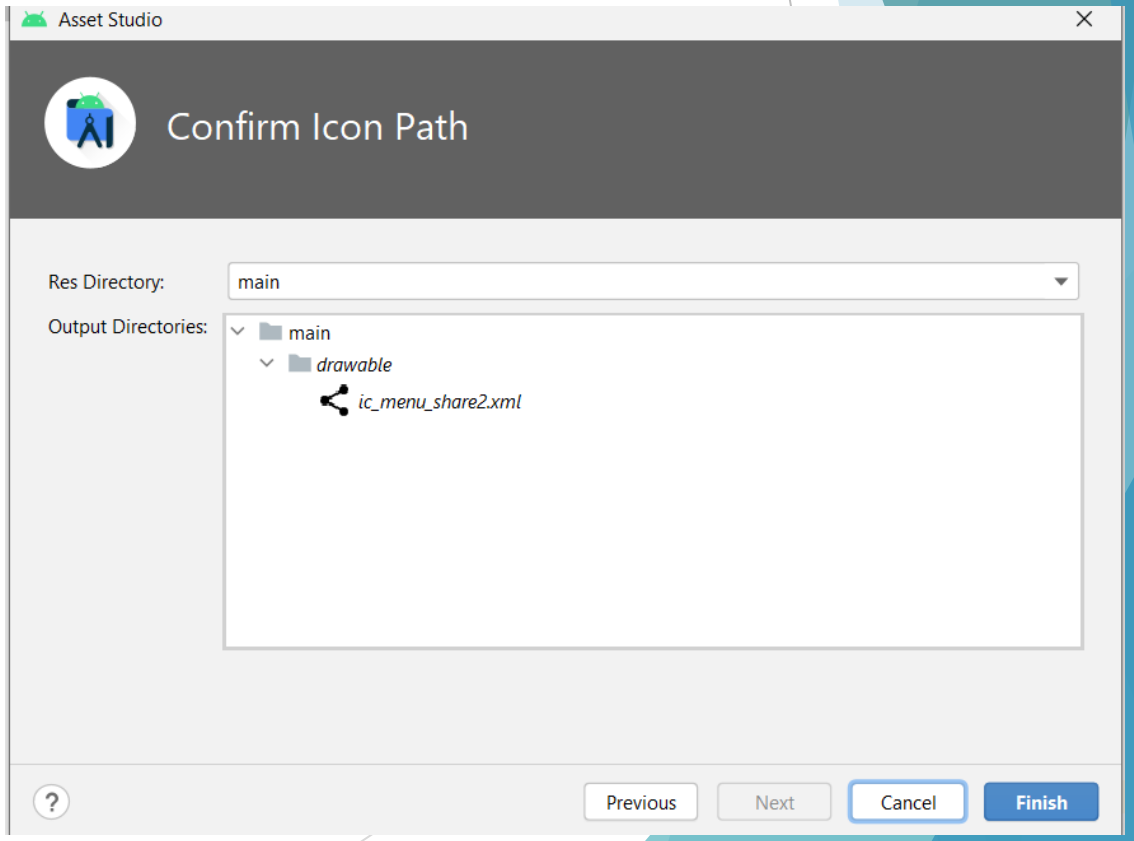




Navegación

- Vamos a coger un icono de la galería por ejemplo el icono de share.



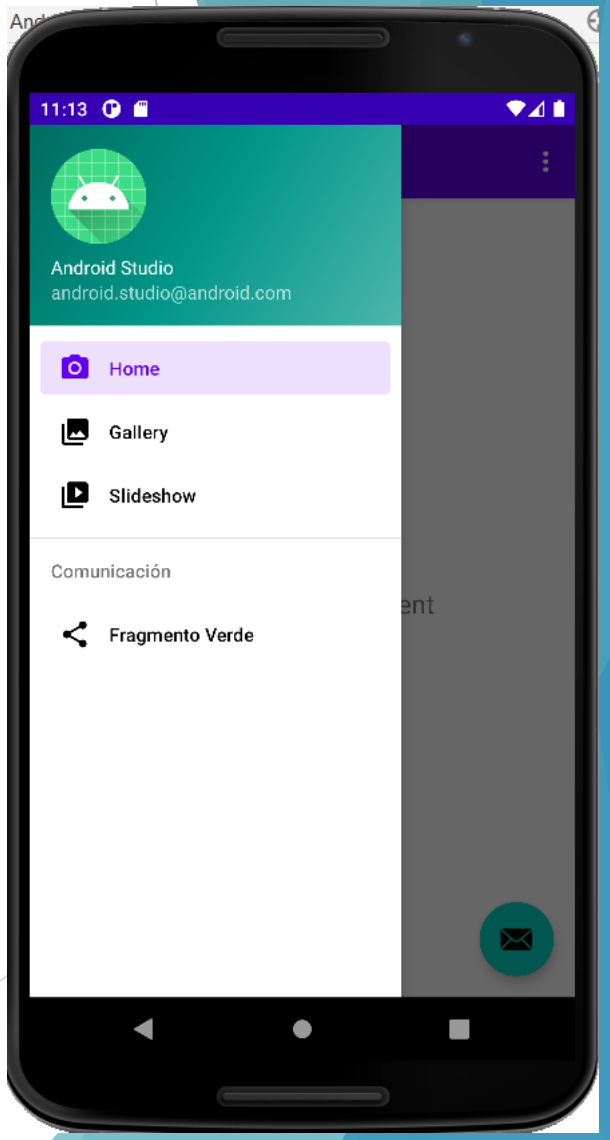


Navegación

- En el archivo activity_main_drawer.xml pondremos en la opción icon el archivo que hemos generado en el drawable.
- El código quedará así:

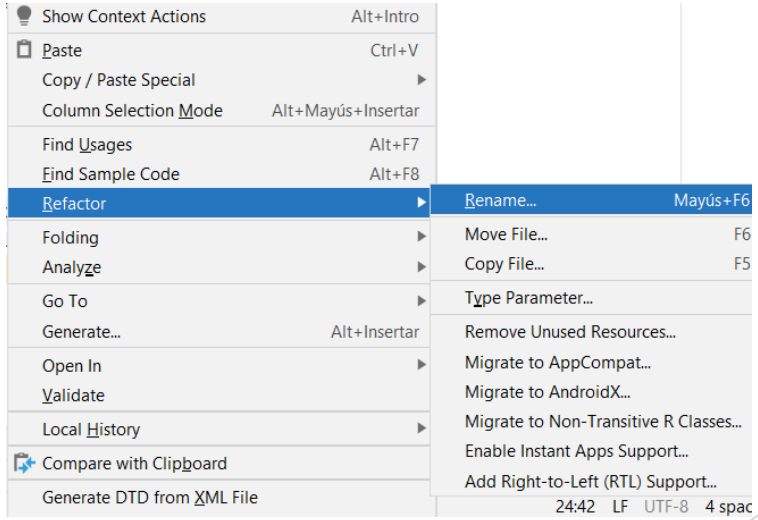
```
<item android:title="Comunicación">
    <menu>
        <item
            android:id="@+id/fragmentVerde"
            android:icon="@drawable/ic_menu_share2"
            android:title="Fragmento Verde" />
    </menu>
</item>
```

- Y el resultado será:
- Si necesitásemos cambiar el texto solamente tendríamos que modificar el campo title. En vez de Fragmento Verde vamos a poner compartir.



Navegación

- Recordar que si cambiamos el id en los elementos del activity_main_drawer.xml, también hay que cambiarlo en el MainActivity.java.
- Hay una opción pulsando el botón derecho del ratón encima del id que queremos modificar. Es la opción refactor y nos pide el nombre nuevo y lo cambia en los dos archivos.
- El resultado será:



Android

- app
  - manifests
  - java
    - com.example.ejemplonavigatordrawer
      - ui
        - gallery
        - home
        - slideshow
      - FragmentVerde
      - MainActivity
    - com.example.ejemplonavigatordrawer (androidTest)
    - com.example.ejemplonavigatordrawer (test)
  - java (generated)
  - res
    - drawable
      - ic_launcher_background.xml
      - ic_launcher_foreground.xml (v24)
      - ic_menu_camera.xml
      - ic_menu_gallery.xml
      - ic_menu_share.xml
      - ic_menu_share2.xml
      - ic_menu_slideshow.xml
      - side_nav_bar.xml
    - layout
      - activity_main.xml
      - app_bar_main.xml
      - content_main.xml
      - fragment_gallery.xml
      - fragment_home.xml
      - fragment_slideshow.xml
      - fragment_verde.xml
      - nav_header_main.xml
    - menu
      - activity_main_drawer.xml
      - main.xml

leshow.xml | v24\ic_launcher_foreground.xml | ic_launcher_background.xml | activity_main.xml | activity_main_drawer.xml | MainActivity.java

Code | Split | Design

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:showIn="navigation_view">

    <group android:checkableBehavior="single">
        <item
            android:id="@+id/nav_home"
            android:icon="@drawable/ic_menu_camera"
            android:title="Home" />
        <item
            android:id="@+id/nav_gallery"
            android:icon="@drawable/ic_menu_gallery"
            android:title="Gallery" />
        <item
            android:id="@+id/nav_slideshow"
            android:icon="@drawable/ic_menu_slideshow"
            android:title="Slideshow" />

    </group>
    <item android:title="Comunicación">
        <menu>
            <item
                android:id="@+id/nav_compartir"
                android:icon="@drawable/ic_menu_share2"
                android:title="Compartir" />
        </menu>
    </item>
</menu>
```

menu

Your anti-virus program might be impacting your build performance. Android Studio checked the...

Actions | Details

TODO | Problems | Terminal | Logcat | Build | Profiler | Run | App Inspection | Event Log | Layout Inspector

Android

- app
  - manifests
  - java
    - com.example.ejemplonavigatordrawer
      - ui
        - gallery
        - home
        - slideshow
      - FragmentVerde
      - MainActivity
    - com.example.ejemplonavigatordrawer (androidTest)
    - com.example.ejemplonavigatordrawer (test)
  - java (generated)
  - res
    - drawable
      - ic_launcher_background.xml
      - ic_launcher_foreground.xml (v24)
      - ic_menu_camera.xml
      - ic_menu_gallery.xml
      - ic_menu_share.xml
      - ic_menu_share2.xml
      - ic_menu_slideshow.xml
      - side_nav_bar.xml
    - layout
      - activity_main.xml
      - app_bar_main.xml
      - content_main.xml
      - fragment_gallery.xml
      - fragment_home.xml
      - fragment_slideshow.xml
      - fragment_verde.xml
      - nav_header_main.xml
    - menu
      - activity_main_drawer.xml
      - main.xml

leshow.xml | v24\ic_launcher_foreground.xml | ic_launcher_background.xml | activity_main.xml | activity_main_drawer.xml | MainActivity.java

```
31          setSupportActionBar(binding.appBarMain.toolbar);
32          binding.appBarMain.fab.setOnClickListener(new View.OnClickListener() {
33              @Override
34              public void onClick(View view) {
35                  Snackbar.make(view, text: "Replace with your own action", Snackbar.LENGTH_LONG)
36                          .setAction( text: "Action", listener: null).show();
37              }
38          });
39          DrawerLayout drawer = binding.drawerLayout;
40          NavigationView navigationView = binding.navView;
41          // Passing each menu ID as a set of Ids because each
42          // menu should be considered as top level destinations.
43          mAppBarConfiguration = new AppBarConfiguration.Builder(
44                  R.id.nav_home, R.id.nav_gallery, R.id.nav_slideshow, R.id.nav_compartir)
45                  .setOpenableLayout(drawer)
46                  .build();
47          NavController navController = Navigation.findNavController( activity: this, R.id.nav_host_fragment_content_ma
48          NavigationUI.setupActionBarWithNavController( activity: this, navController, mAppBarConfiguration);
49          NavigationUI.setupWithNavController(navigationView, navController);
50      }
51
52      @Override
53      public boolean onCreateOptionsMenu(Menu menu) {
54          // Inflate the menu; this adds items to the action bar if it is present.
55          getMenuInflater().inflate(R.menu.main, menu);
56          return true;
57      }
58
59      @Override
60      public boolean onSupportNavigateUp() {
61          NavController navController = Navigation.findNavController( activity
62          return NavigationUI.navigateUp(navController, mAppBarConfiguratio
63                  || super.onSupportNavigateUp();
```

Your anti-virus program might be impacting your build performance. Android Studio checked the...

Actions ▾ Details

Ahora os toca trabajar a vosotros

Personalizar la cabecera del menú como vosotros penséis que se adapta más al programa que os gustaría hacer.

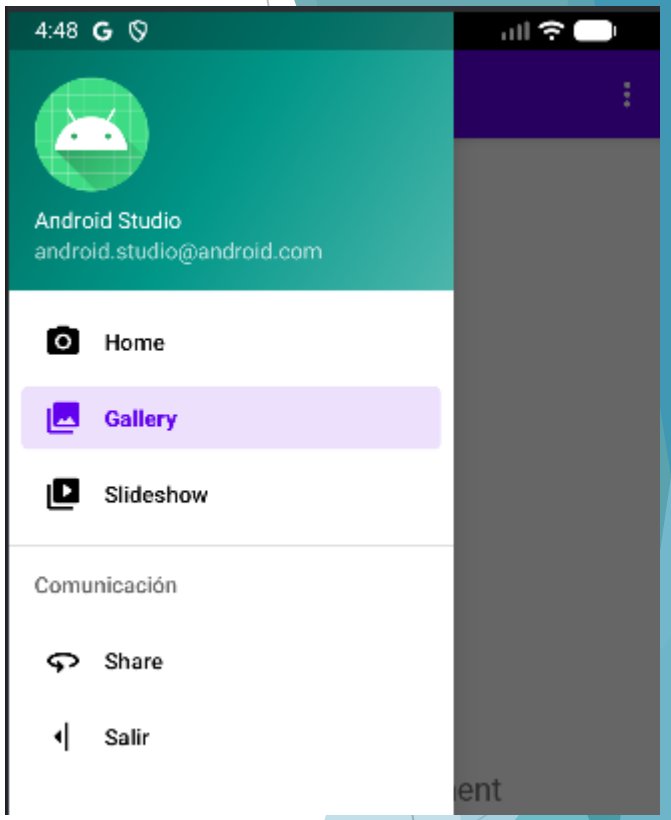
Modificar las opciones de menú que han aparecido por defecto, adaptándolas a las opciones de vuestro programa. Modificareis imagen y texto.

Añadir dos nuevas opciones al menú que sea la opción listado y la opción salir.

Navegación

- Podemos introducir acciones en el menú de navegación, por ejemplo, la de salir.
- Después definimos en el ActivityMain.java un listener para reaccionar a su pulsación, como un item del menú de navegación.

```
navigationView.getMenu().findItem(R.id.nav_exit).setOnMenuItemClickListener( MenuItem item -> {
    finishAffinity();   // Cierra completamente la app
    System.exit( status: 0);
    return true;
});
```



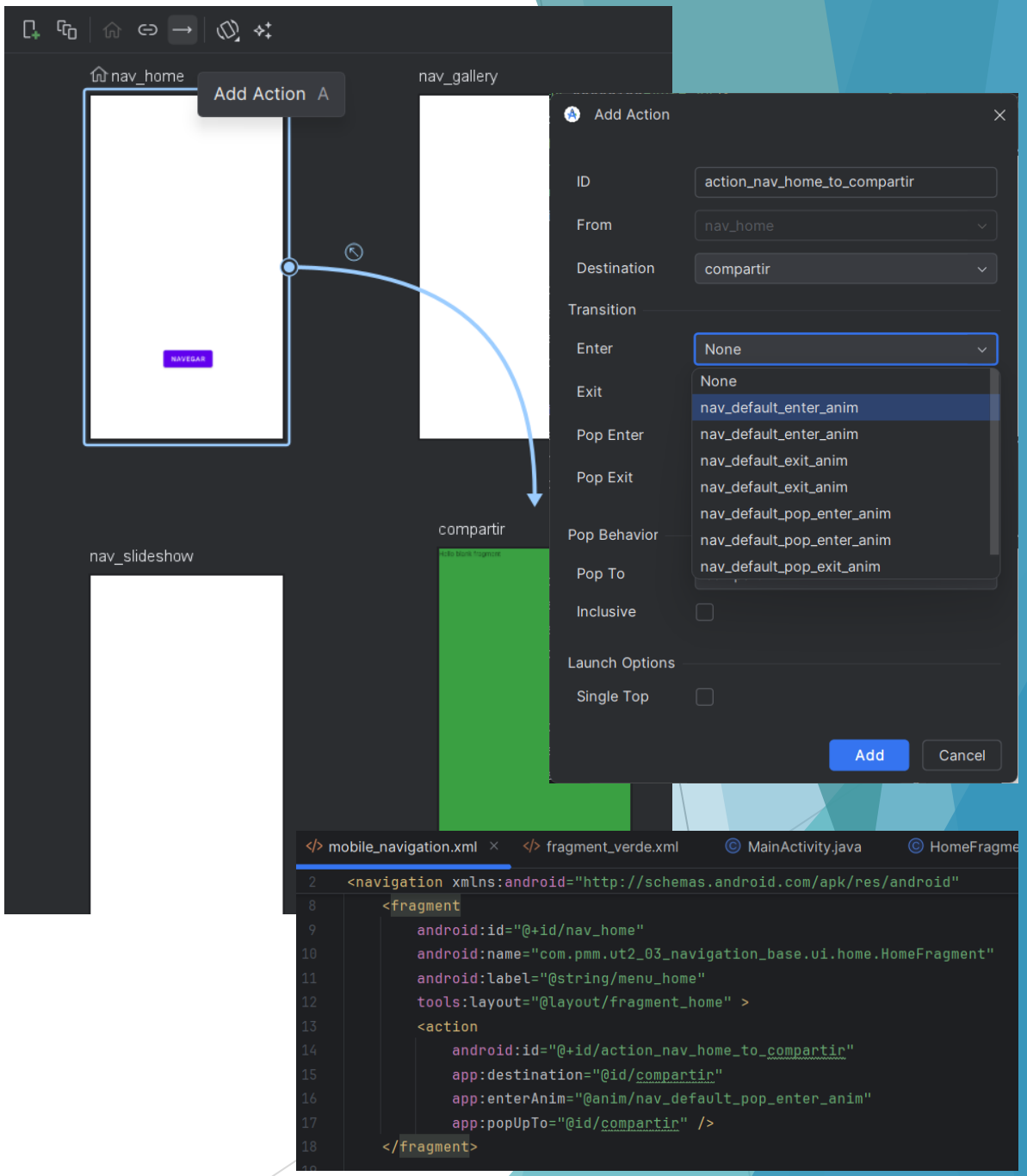
- Se hacen dos llamadas:
  - finishAffinity() → Cierra todas las activities
  - System.exit(0) → Cierra el proceso de ejecución (no necesario)

Navegación

- Se pueden establecer acciones de navegación entre fragmentos desde el menú de navegación (mobile_navigation.xml).
- Después de definir la acción, podemos incluir en el correspondiente Fragment.java (el de Home en el ejemplo) la acción de navegación al pulsar un botón.

```
Button btnNavigate= binding.btnNav;
btnNavigate.setOnClickListener( View view -> {
    NavController navController =
            NavHostFragment.findNavController( fragment: HomeFragment.this);

    navController.navigate(R.id.action_nav_home_to_compartir);
});
```

- El resultado será que sustituiremos el fragmento origen por el destino.



Asignación permisos APP

Permisos APP

- Los permisos de la app ayudan a mejorar la privacidad del usuario, ya que protegen el acceso a lo siguiente:
  - Datos restringidos, como el estado del sistema y la información de contacto del usuario
  - Acciones restringidas, como conectarte a un dispositivo vinculado y grabar audio.
- Existen dos tipos de permisos principales que podemos utilizar:
  - Permisos en el momento de la instalación. Dentro de estos tendremos:
    - Permisos normales
    - Permisos de firma
  - Permisos de tiempo de ejecución
  - Permisos especiales

Permisos de instalación

- Otorgan a la aplicación acceso limitado a los datos restringidos
- Permiten que realice acciones restringidas que casi no afectan al sistema o a otras apps.
- Cuando declaras permisos en el momento de la instalación en la app, el sistema le otorga automáticamente los permisos cuando el usuario la instala.
- Los permisos normales permiten el acceso a los datos y las acciones que se extienden más allá de la zona de pruebas de tu app. Presentan muy poco riesgo para la privacidad del usuario y el funcionamiento de otras apps.
- Los permisos de firma se refieren a la firma de las aplicaciones con un certificado.

Permisos en tiempo de ejecución

- Se conocen también como permisos peligrosos.
- Le otorgan a tu app acceso adicional a datos restringidos y permiten que realice acciones restringidas que afectan de manera más considerable el sistema y otras apps.
- Debes solicitar permisos de tiempo de ejecución en tu app antes de poder acceder a los datos restringidos o realizar acciones restringidas.
- Algunos ejemplos de datos privados del usuario incluyen la ubicación y la información de contacto, así como el acceso al micrófono o la cámara.

Permisos especiales

- Los permisos especiales corresponden a operaciones particulares de la app.
- Solo la plataforma y los OEM (Fabricantes de Producto Original) pueden definir permisos especiales.
- Además, la plataforma y los OEM suelen definir permisos especiales cuando quieren proteger el acceso a acciones particularmente importantes.

Configuración permisos APP

- En un inicio los permisos se configuraban únicamente en el AndroidManifest.
- En la actualidad también hay que ubicarlos en el mismo archivo entre las etiquetas <manifest> y <aplication>
- Una vez ubicados ahí, pondremos la etiqueta <uses-permission> y nos aparecerán un montón de opciones que podemos solicitar al usuario para que nos de permiso.
- Por ejemplo, vamos a solicitar permiso para enviar SMS y para acceder al almacenamiento de datos (fotos) del dispositivo.
- El archivo AndroidManifest quedaría así:

ruebaPermisos 〉 app 〉 src 〉 main 〉 AndroidManifest.xml

app | Nexus 6 API 30

Android

- app
  - manifests
    - AndroidManifest.xml
  - java
    - com.example.pruebapermisos
      - MainActivity
    - com.example.pruebapermisos (androidTest)
    - com.example.pruebapermisos (test)
  - res
- Gradle Scripts

activity_main.xml | MainActivity.java | AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.pruebapermisos">

    <uses-permission android:name="android.permission.SEND_SMS"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"></uses-permission>

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="PruebaPermisos"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.PruebaPermisos">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

manifest 〉 application

Text | Merged Manifest

Configuración permisos APP

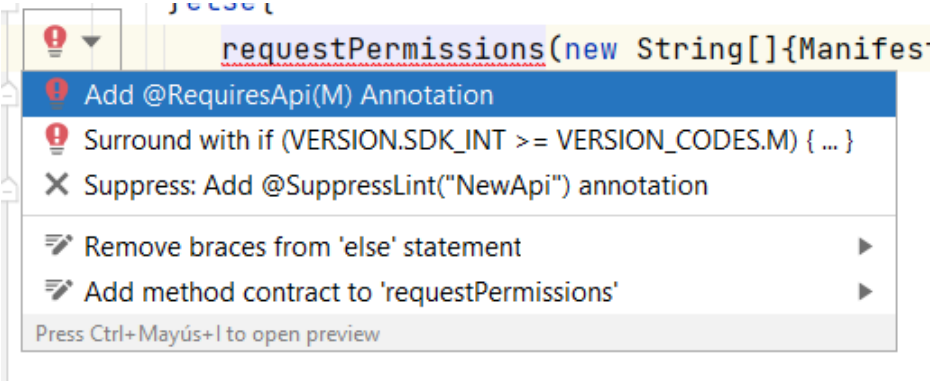
- Lo siguiente que tenemos que hacer es comprobar en el MainActivity.java si ya tenemos esos permisos o no.
- Para ello vamos a crear un método que se llame verificarPermisos que vamos a ejecutar en el método onCreate.
- Nos dará un error porque es un método que no está creado. Con la ayuda de errores le daremos a crear método.
- Para comprobarlo utilizaremos un método que se llama ContextCompat.checkSelfPermission(). Como parámetros pondremos el contexto (this) y el permiso que queremos revisar, en este caso revisaremos el primero que hemos puesto que era el de enviar mensajes.

Configuración permisos APP

- El método va a devolver uno de estos dos valores PERMISSION_GRANTED (0) o PERMISSION_DENIED (-1), del PackageManager. Como nos va a devolver un número lo almacenaremos en una variable que llamaremos permisoSMS de tipo entero.
- Lo siguiente será comprobar con un if que valor tiene, es decir si tenemos el permiso concedido o denegado.
- Si tenemos el permiso, vamos a poner un Toast indicándolo.
- Si no lo tenemos le solicitaremos al usuario que nos de permiso.
- Para ello usaremos un método que es requestPermissions.

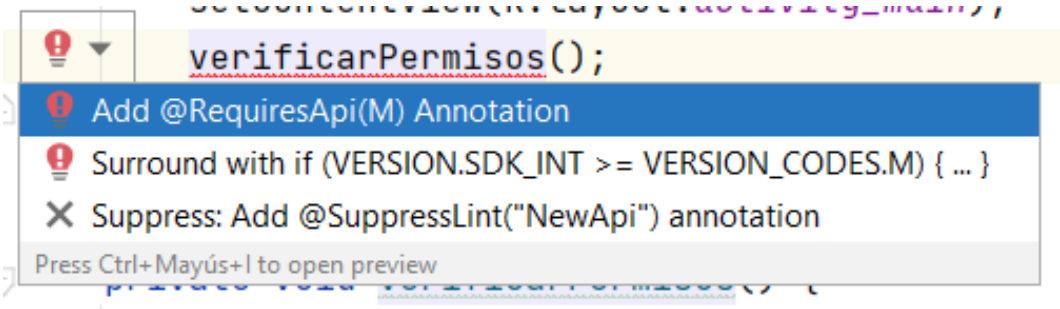
Configuración permisos APP

- Como parámetros tendrá un array que almacenará los permisos que se están solicitando al usuario y como segundo parámetro necesita la respuesta que nos va a dar el usuario a la petición.
- Tenemos que generar una variable de tipo entero que tendrá el valor que nosotros indiquemos para saber si el usuario aceptó o no el darnos el permiso. A la variable la vamos a llamar CODIGO_RESPUESTA.
- El método va a dar error porque se ha implementado para versiones nuevas por lo que tendremos que dar a la ayuda y generar la notación.



Configuración permisos APP

- También lo solicita para la invocación del método en el onCreate



- El MainActivity quedaría así:

PruebaPermisos > app > src > main > java > com > example > pruebapermisos > MainActivity > onCreate

app | Nexus 6 API 30

Android

- app
  - manifests
    - AndroidManifest.xml
  - java
    - com.example.pruebapermisos
      - MainActivity
    - com.example.pruebapermisos (androidTest)
    - com.example.pruebapermisos (test)
  - res
- Gradle Scripts

activity_main.xml | MainActivity.java | AndroidManifest.xml

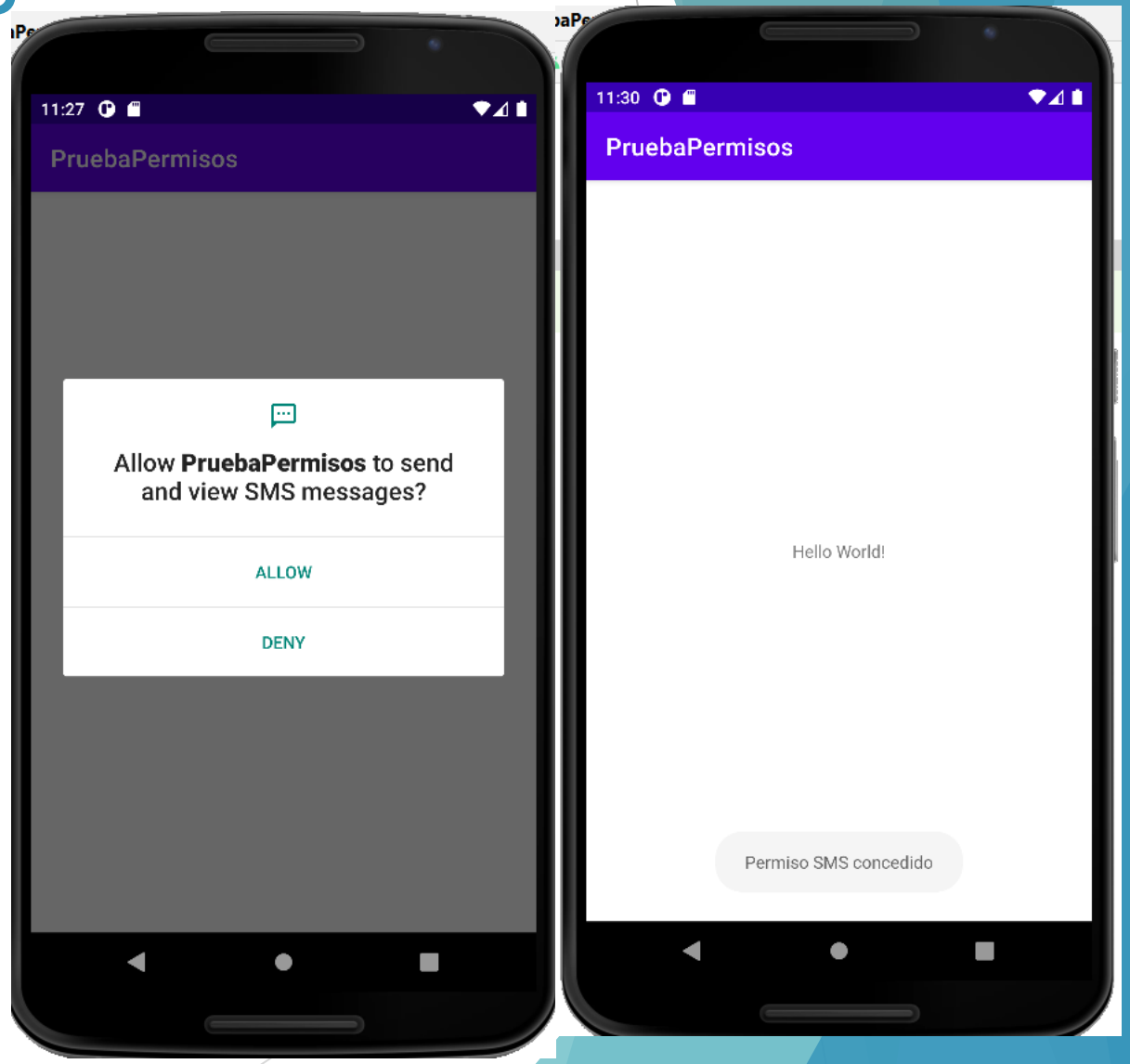
```java
package com.example.pruebapermisos;

import ...

public class MainActivity extends AppCompatActivity {
    int CODIGO_RESPUESTA = 200;
    @RequiresApi(api = Build.VERSION_CODES.M)
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        verificarPermisos();
    }

    @RequiresApi(api = Build.VERSION_CODES.M)
    private void verificarPermisos() {
       int permisoSMS = ContextCompat.checkSelfPermission( context: this, Manifest.permission.SEND_SMS);
       if (permisoSMS == PackageManager.PERMISSION_GRANTED){
            Toast.makeText( context: this,  text: "Permiso SMS concedido", Toast.LENGTH_SHORT).show();
        }else{
           requestPermissions(new String[]{Manifest.permission.SEND_SMS},CODIGO_RESPUESTA);
       }

    }
}
```

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Event Log | Layout Inspec

Configuración permisos APP

- Si ejecutamos la aplicación aparecerá:
- Si le damos a aceptar (ALLOW) el valor 200

se nos almacenará en nuestro archivo y cuando volvamos a ejecutar la aplicación detectará que ya le hemos dado permiso y no lo volverá a solicitar.



Ahora te toca trabajar a ti

- Realiza las comprobaciones del permiso de envío de SMS que hemos definido
- Averigua cuales son los permisos que hay que solicitar para tener conexión a internet, acceder a los contactos de la agenda y a la ubicación del usuario.
- Realiza las comprobaciones de los tres permisos anteriores para que, si no tenemos acceso, se lo solicitemos al usuario.
- De los tres tipos de permisos que hemos visto ¿a cuál de ellos pertenecerían estos permisos? Razona la respuesta.

Configuración permisos APP

- Es posible solicitar varios permisos en una sola acción, enumerándolos:

```
requestPermissions(new String[]{Manifest.permission.ACCESS_MEDIA_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION}, CODIGO_PERMISOS_MEDIA);
```

- Y podemos determinar acciones en función de la respuesta del usuario a la solicitud de permisos en un método onRequestPermissionsResult ejecutado automáticamente tras solicitar los permisos.
- Se identifica la solicitud en particular (si hubiera diferentes en la app) usando el mismo código de permiso que identificamos en la llamada a requestPermissions

```
@Override
public void onRequestPermissionsResult(int requestCode,
                                       @NonNull String[] permissions,
                                       @NonNull int[] grantResults) {

    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    if (requestCode == CODIGO_PERMISO_SMS) {
        if (grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            // Usuario aceptó
```

Configuración permisos APP

- En el caso de los permisos especiales, no tendremos opción para recopilarlos, ni siquiera, aunque se los solicitemos al usuario.

```
<uses-permission android:name="android.permission.MEDIA_CONTENT_CONTROL"/>
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xm
    android:fullBackupContent="@xml/
    android:icon="@mipmap/ic_launche
    android:label="UT2_03_Permisos_B
    android:roundIcon="@mipmap/ic_la
    android:supportsRtl="true"
    android:theme="@style/Theme.UT2_
    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="an

            <category android:name="
        </intent-filter>
    </activity>
```

Permission is only granted to system apps Toggle info (Ctrl+F1)

Suppress: Add tools:ignore="ProtectedPermissions" attribute Alt+Mayús+Intro More actions... Alt+Intro

MEDIA_CONTENT_CONTROL

Added in API level 19

public static final String MEDIA_CONTENT_CONTROL

Allows an application to know what content is playing and control its playback.

Not for use by third-party applications due to privacy of media consumption

Constant Value: "android.permission.MEDIA_CONTENT_CONTROL"

`MEDIA_CONTENT_CONTROL` on developer.android.com

< Android API 36.0, extension level 17 Platform > (android.jar)

Servicios de una APP y Sensores

Servicios de una APP

- Un servicio es un componente de una aplicación que puede realizar operaciones de larga ejecución en segundo plano y que no proporciona una interfaz de usuario.
- Otro componente de la aplicación puede iniciar un servicio y continuar ejecutándose en segundo plano aunque el usuario cambie a otra aplicación.
- Además, un componente puede enlazarse con un servicio para interactuar con él e incluso realizar una comunicación entre procesos

Servicios de una APP

- Tenemos tres tipos diferentes de servicios:
  - Primer plano
    - Un servicio en primer plano realiza una operación que el usuario puede notar. Por ejemplo, una aplicación de audio usa un servicio en primer plano para reproducir una pista de audio.
    - Los servicios en primer plano deben mostrar una notificación. Estos servicios continúan ejecutándose incluso si el usuario deja de interactuar con la aplicación.

Servicios de una APP

- Segundo plano
  - Realiza una operación que el usuario no nota directamente. Por ejemplo, si una aplicación usa un servicio para comprimir su almacenamiento, suele tratarse de un servicio en segundo plano.
- Enlace
  - Ofrece una interfaz cliente-servidor que permite que los componentes interactúen con el servicio, envíen solicitudes, reciban resultados e incluso lo hagan en distintos procesos con la comunicación entre procesos (IPC).
  - Un servicio de enlace se ejecuta solamente mientras otro componente de aplicación está enlazado a él. Se pueden enlazar varios componentes con el servicio a la vez, pero el servicio se destruye cuando todos ellos se desenlazan.

Servicios de una APP

- Existen dos tipos de servicios en android:
  - Aquel que se inicia explícitamente a través del método startService() (Started Service)
  - y el que se liga a un componente con bindService() (Bound Service).
- El started service realiza una operación determinada y se cierra al momento de finalizarla.
- Por otro lado, la esperanza de vida de un bound service ó servicio ligado depende de la vida del elemento que lo generó.

Servicios de una APP

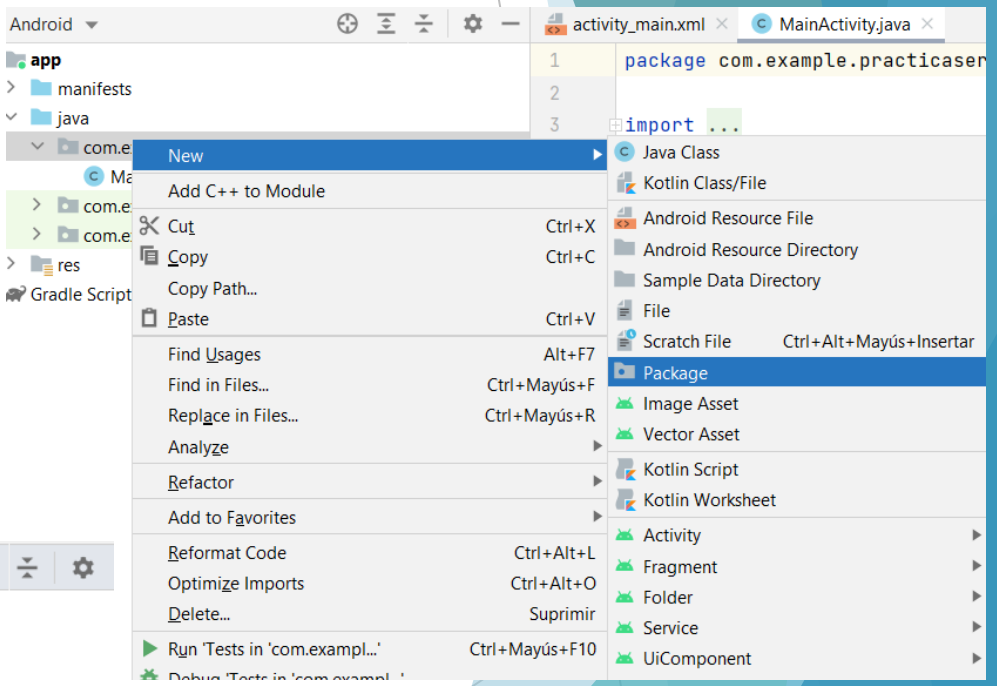
- Los métodos que controlan el flujo de un servicio son:
  - onCreate(): Se ejecuta cuando el servicio está creado en memoria. Si el servicio ya está activo, entonces se evita de nuevo su llamada.
  - onStartCommand():Método que ejecuta las instrucciones del servicio. Se llama solo si el servicio se inició con startService().
  - onBind(): Solo se ejecuta si el servicio fue ligado con bindService() por un componente. Retorna una interfaz Java de comunicación del tipo IBinder. Este método siempre debe llamarse, incluso dentro de los started services, los cuales retornan null.
  - onDestroy(): Se llama cuando el servicio está siendo destruido. Importantísimo que dentro de este método detengas los hilos iniciados.

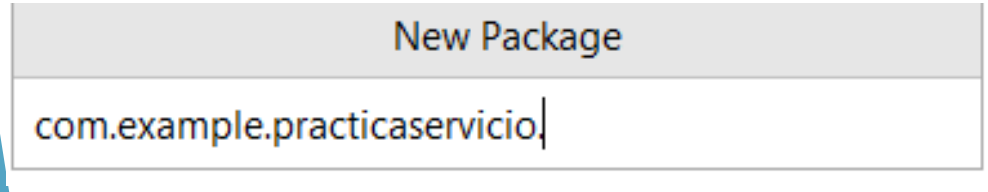
Servicios de una APP

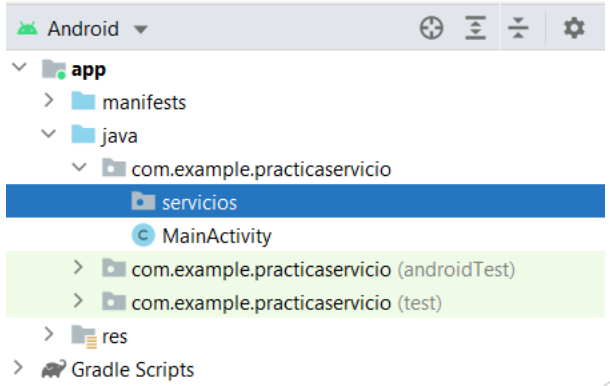
- A pesar de los tres tipos de servicios que hay, generalmente un servicio se va a utilizar para realizar tareas en segundo plano y sin interfaz de usuario.
- Algunas acciones útiles podrían ser:
  - Sincronizar aplicaciones con la nube.
  - Reproducir música sin tener que utilizar la interfaz.
  - Almacenar información en base de datos.

Servicios de una APP

- Vamos a crear un servicio en una aplicación Android Studio.
- Nuestra aplicación se va a llamar Práctica Servicio y como siempre la vamos a crear como un Empty Activity.
- Para que quede todo más ordenado, dentro de nuestro proyecto, vamos a crear una carpeta de servicio.
- Para ello, pulsaremos botón derecho del ratón sobre com.example.practicaservicio y le daremos a new/package y crearemos el directorio.

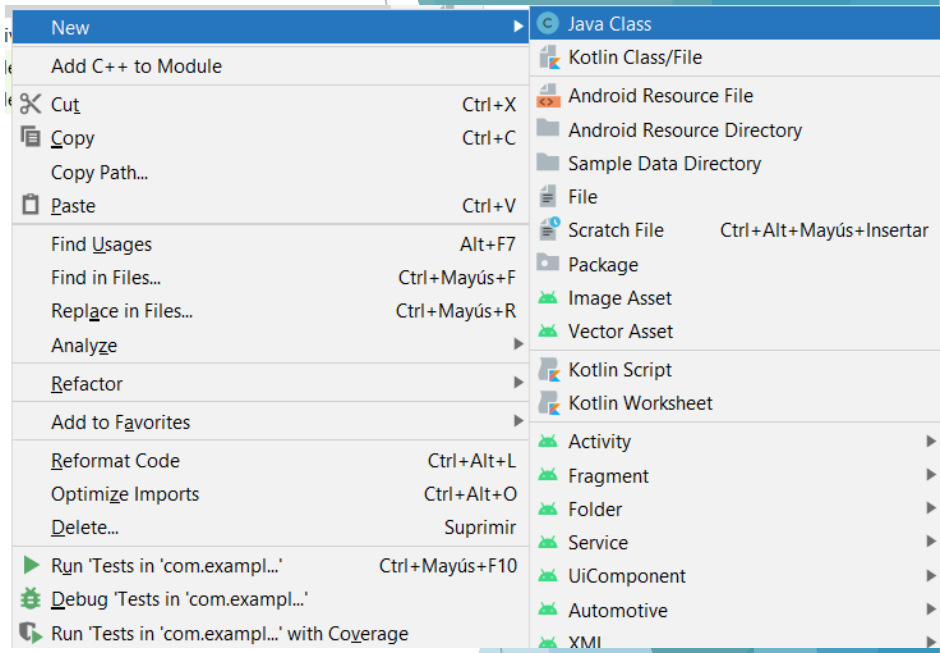


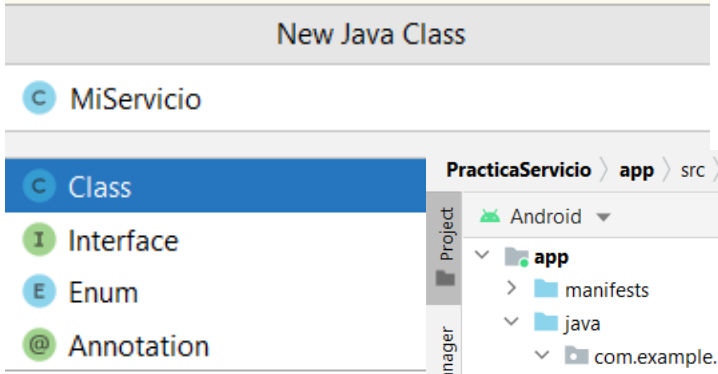


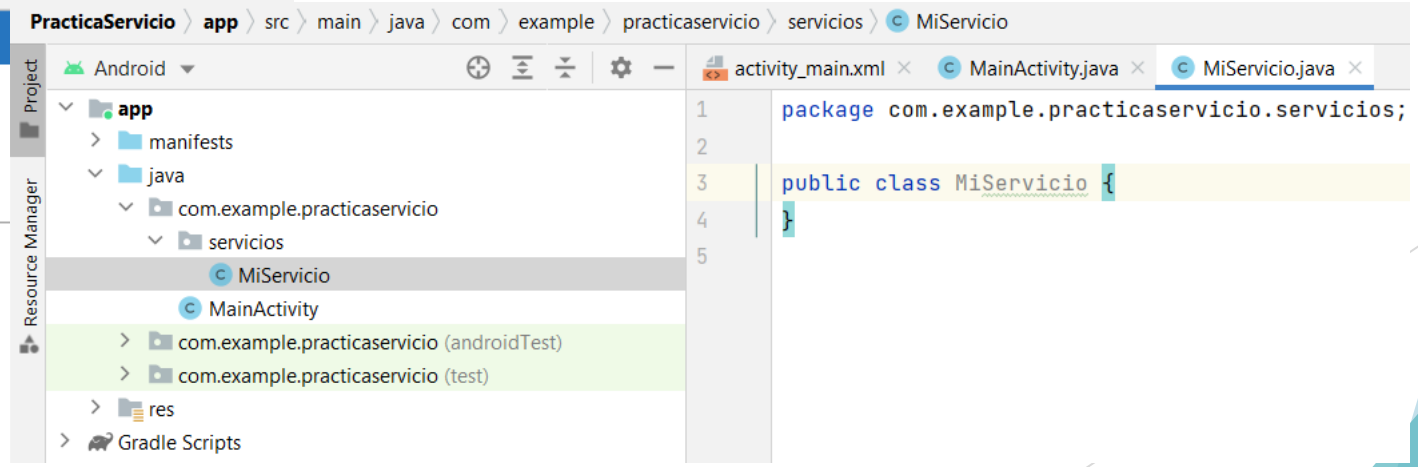


Servicios de una APP

- Dentro de servicios vamos a crear una clase java que vamos a llamar Mi Servicio.
- Para ello pulsaremos sobre la carpeta servicios botón derecho New/JavaClass.







Servicio de una APP (GPS)

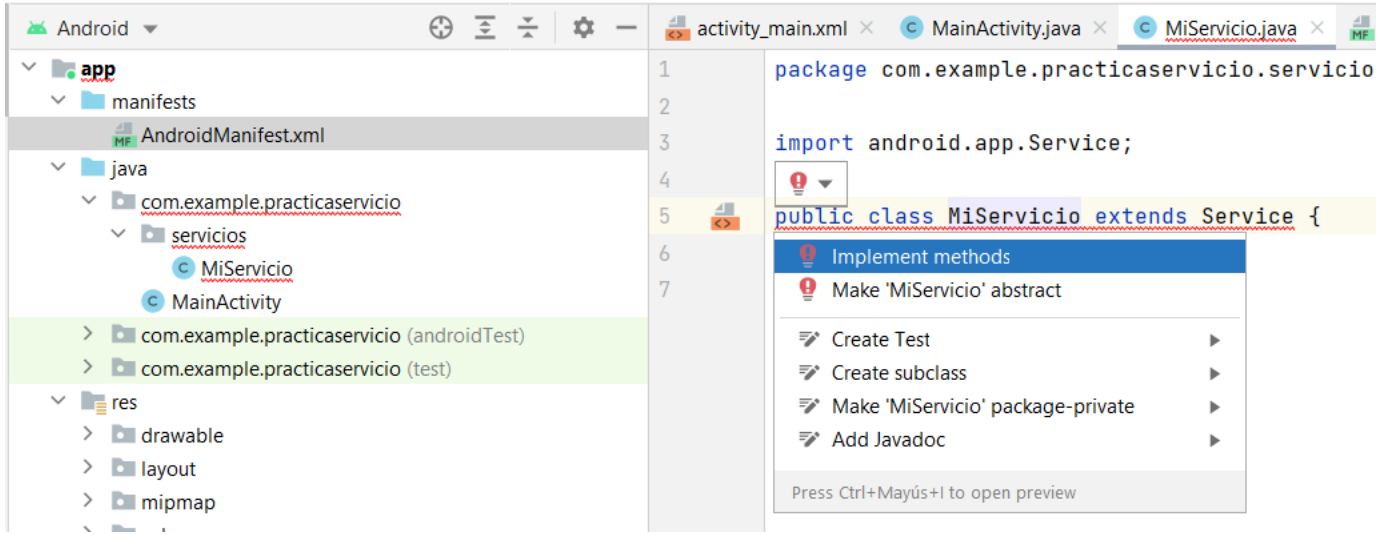
- Una vez creada nuestra clase, tendremos que declararla en nuestro archivo Manifest como un servicio, porque si no nuestro programa no lo va a reconocer como tal.
- Dentro de la etiqueta aplication y al mismo nivel que los activity, declararemos los servicios.
- Lo declaramos de la siguiente manera:

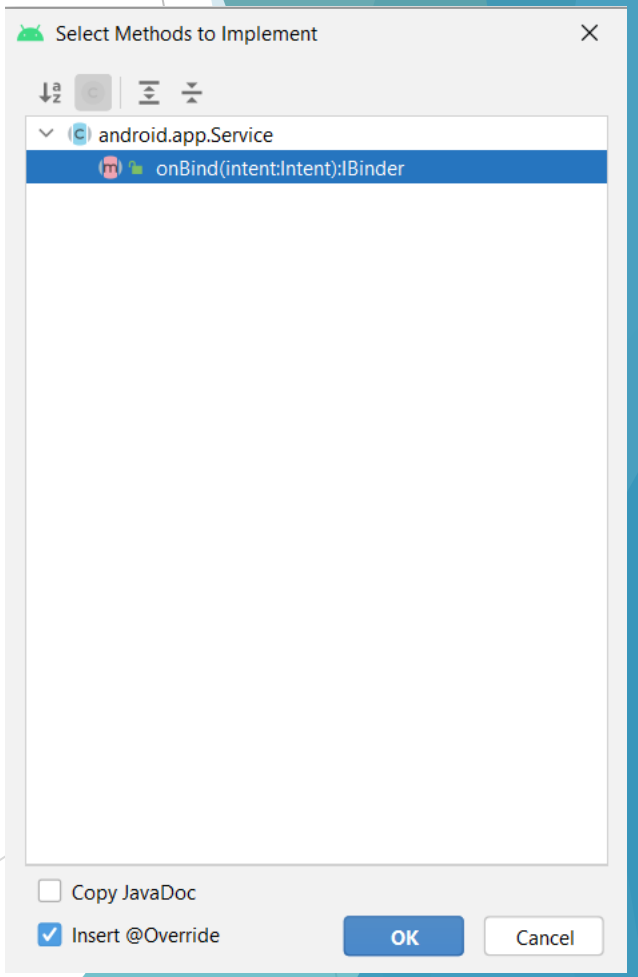
```
<service android:name="com.example.practicaservicio.servicios.MiServicio"/>
```

- Al declarar el servicio nos aparece un error porque no hemos modificado el archivo MiServicio.java.
- Para que nos lo reconozca como servicio pondremos lo siguiente:

Servicio de una APP (GPS)

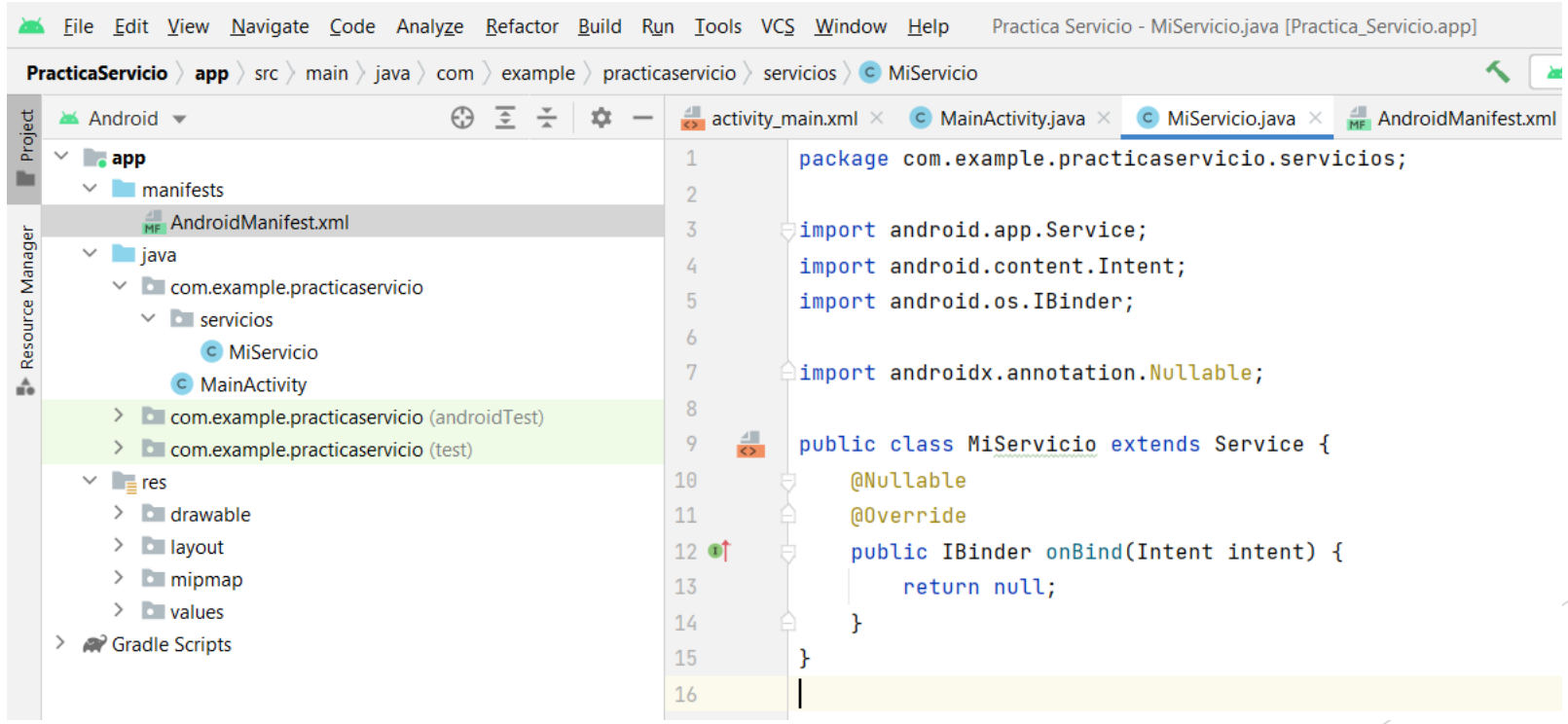
- Pondremos extends Service y nos importará la clase Service android.app.
- Nos aparece error porque necesitamos incluir los métodos. Si pulsamos la ayuda nos lo generará automáticamente.





Servicio de una APP (GPS)

- Lo que finalmente nos queda es la estructura básica del servicio para implementarlo después, según lo que necesitemos.
- Si volvemos al Manifest, veremos que el error ha desaparecido.



Servicio de una APP (GPS)

- Nos faltaría por añadir un método más. Para ello vamos a crear una variable contexto.
- La variable de tipo contexto es un objeto que te permite hacer diferentes operaciones como conectarse a una base de datos o utilizar gestores de contenido.
- Esa variable la utilizaremos como atributo en el método MiServicio.
- El código, por tanto, quedaría de la siguiente manera:

acticaServicio 〉 app 〉 src 〉 main 〉 java 〉 com 〉 example 〉 practicaservicio 〉 servicios 〉 MiServicio

app | Nexus 6 API 30

activity_main.xml | MainActivity.java | MiServicio.java | AndroidManifest.xml

```java
package com.example.practicaservicio.servicios;

import android.app.Service;
import android.content.Context;
import android.content.Intent;
import android.os.IBinder;

import androidx.annotation.Nullable;

public class MiServicio extends Service {

    private Context ctx;

    public MiServicio(Context c){
        super(); //para que se ejecute el constructor del servicio original.
        this.ctx = c;
    }
    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Event Log | Layout Inspector

Gradle sync finished in 2 m 56 s 937 ms (today 18:50)

13:1 CRLF UTF-8 4 spaces

Servicio de una APP (GPS)

- Lo siguiente que haremos será acceder al MainActivity.java para crear el servicio.
- Vamos a importar la clase servicio que hemos creado. Para ello pondremos la siguiente línea de código

import com.example.practicaservicio.servicios.MiServicio;

- Después crearemos el objeto dentro del onCreate. Para ello pondremos la siguiente línea de código:

MiServicio servicio = new MiServicio(getApplicationContext());

- De esta manera obtenemos el contexto de la actividad actual y creamos el objeto.

acticaServicio > app > src > main > java > com > example > practicaservicio > MainActivity

app | Nexus 6 API 30

activity_main.xml | MainActivity.java | MiServicio.java | AndroidManifest.xml

```java
package com.example.practicaservicio;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import com.example.practicaservicio.servicios.MiServicio;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        MiServicio servicio = new MiServicio(getApplicationContext());
    }
}
```

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Event Log | Layout Inspector

Gradle sync finished in 2 m 56 s 937 ms (today 18:50)

16:2 LF UTF-8 4 spaces

Servicio de una APP (GPS)

- Con este código tenemos la estructura básica. Lo que vamos a hacer ahora es conseguir las coordenadas GPS a través de un servicio.
- Para poder acceder a las coordenadas GPS necesitamos tener algunos permisos, de Internet y de localización.
- Los añadiremos en el archivo Manifest.

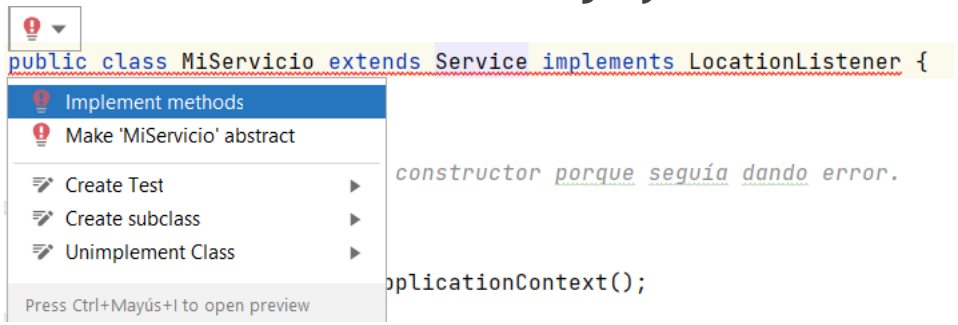
```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.practicaservicio">
    <!--Permisos acceso a la red-->
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
    <!--Permisos acceso al GPS-->
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_LOCATION_EXTRA_COMMANDS"/>
```

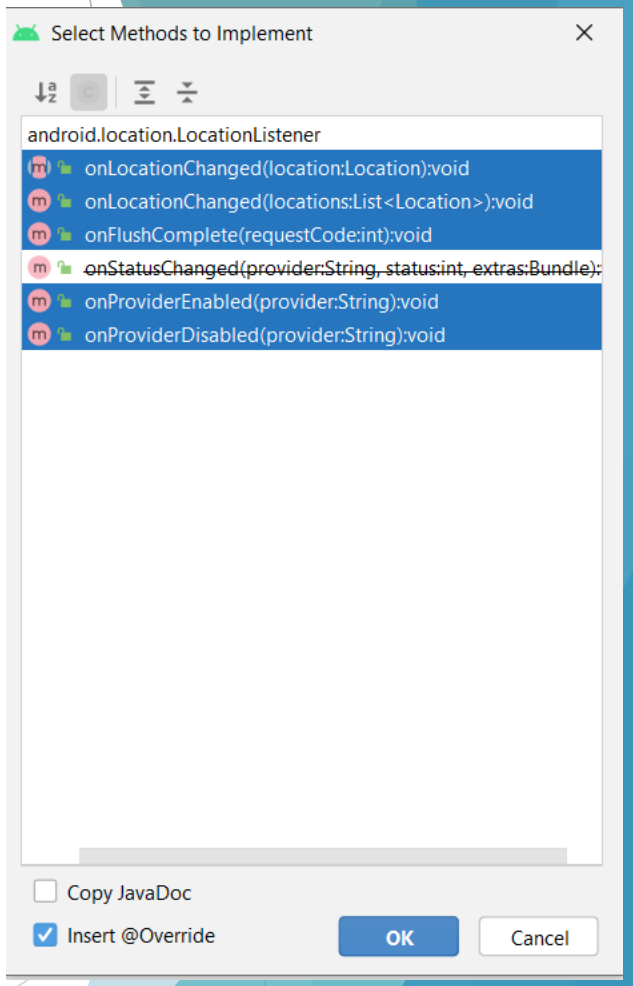
Servicio de una APP (GPS)

- Ahora vamos a acceder de nuevo a nuestro servicio MiServicio.java y vamos a modificarlo.
- Lo primero que haremos será añadir una interface al método que se llama LocationListener que nos va a permitir conectarnos con los servicios de ubicación.

public class MiServicio extends Service implements LocationListener

- Como vemos nos da error y eso es debido a que nos faltan definir algunos métodos adicionales.
- Pulsaremos a la bombilla roja y nos creará automáticamente dichos métodos.





Servicio de una APP (GPS)

- En el método onLocationChanged controlará cada cierto tiempo nuestra posición, ya que podemos estar en movimiento.
- En el método onProviderEnabled y onProviderDisabled nos indica cuando el proveedor está habilitado o deshabilitado.
- Nuestro servicio va a ser simple, ya que lo que haremos será obtener las coordenadas una sola vez y mostrarlas por pantalla.
- Lo siguiente que vamos a hacer es crear una serie de variables que vamos a necesitar.
- Vamos a generar Latitud y Longitud de tipo Double porque pueden tener varios decimales.

Servicio de una APP (GPS)

- Vamos a crear Localización que será de tipo Location. Este tipo de variables pueden almacenar otros valores referentes a la localización como la altitud o la precisión.
- Crearemos también una variable que llamaremos gpsActivo que será de tipo Boolean y nos indicará si el gps está activo o no.
- Por último crearemos un textView que llamaremos texto que es donde escribiremos las coordenadas.
- Vamos a crear un método que se llame setView (ya le estamos utilizando en el ActivityMain) que nos mostrará la ubicación.

```
private Context ctx;
double latitud;
double longitud;
Location localizacion;
boolean gpsActivo;
TextView texto;
```

```
public void setView (View v){
    texto = (TextView)v;
    texto.setText("Coordenadas:"+latitud+", "+ longitud);
}
```

Servicio de una APP (GPS)

- Crearemos otro método para obtener la ubicación.
- Lo llamaremos getLocation y lo vamos a llamar en el momento de crear nuestro servicio en el constructor.
- Antes de continuar, necesitaremos generar una nueva variable del tipo LocationManager que nos permitirá manejar el tipo de conexión.

```
LocationManager locationManager;
```

- Vamos a generar un try-catch y vamos a inicializar la variable locationManager.
- El hacer referencia al contexto nos va a permitir poder acceder a servicios del sistema, en este caso al servicio de ubicación que nos va a permitir acceder a las coordenadas de ubicación.
- Lo siguiente es inicializar la variable booleana gpsActivo para asegurarnos de que esté activo.

Servicio de una APP (GPS)

```
@SuppressLint("MissingPermission")
public void getLocation() {
    try {
        locationManager = (LocationManager) this.ctx.getSystemService(LOCATION_SERVICE);
        gpsActivo=locationManager.isProviderEnabled(LocationManager.GPS_PROVIDER);
    } catch (Exception e) {
    }
    if(gpsActivo)
    {
        locationManager.requestLocationUpdates(locationManager.GPS_PROVIDER
                , minTimeMs: 1000*60
                , minDistanceM: 10, listener: this
                );
    }
}
```

Servicio de una APP (GPS)

- Ahora pediremos la última ubicación conocida.
- En el objeto location que hemos definido anteriormente en las variables (localización), almacenaremos la última ubicación conocida que la obtendrá del proveedor de GPS.

```
localizacion = locationManager.getLastKnownLocation(locationManager.GPS_PROVIDER);
```

- El elemento localización tendrá acceso a diferentes elementos.

```
longitud = localizacion.g();
latitud = localizacio
                                  getLongitude()        double
                                  getLatitude()         double
                                  getAccuracy()         float
                                  getAltitude()         double
.e                                getBearing()          float
le                                getBearingAccuracyDegrees()  float
Binder onBind(Intent              getElapsedRealtimeNanos()    long
rn null;                          getElapsedRealtimeUncertaintyNanos()  double
                                  getSpeed()            float
                                  getSpeedAccuracyMetersPerSecond()  float
le                                getTime()             long
void onLocationChange
```

Servicios de una APP

- Entonces cogeremos los valores de latitud y longitud y los igualaremos a las variables que hemos creado anteriormente.

```
longitud = localizacion.getLongitude();
latitud = localizacion.getLatitude();
```

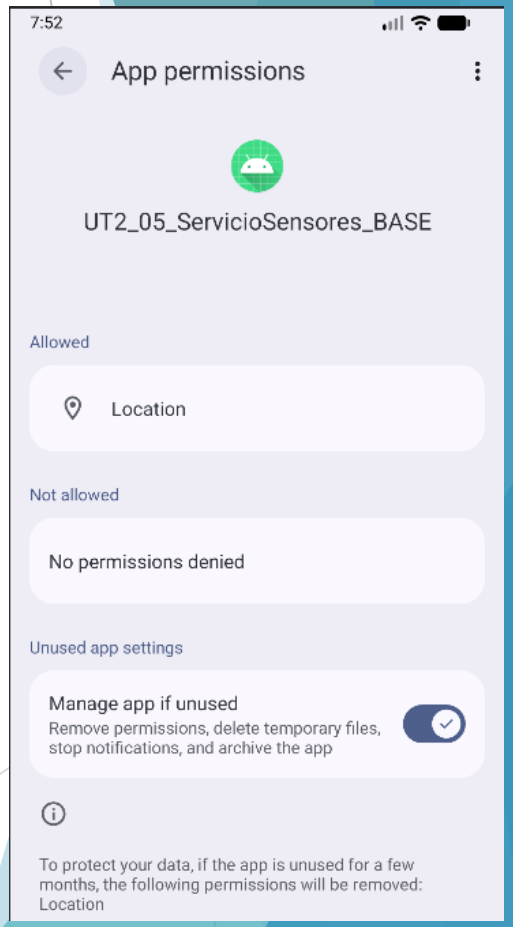
- Con esto ya tendríamos nuestro programa hecho.
- Nos quedaría realizar unos cambios en el activitymain.xml.
- Al TextView que nos aparece le vamos a poner un id que será por ejemplo, textoUbicacion.
- Volvemos al MainActivity.java y tendremos que añadir unas cuantas líneas de código más.

Servicio de una APP (GPS)

- Definiremos una variable de tipo TextView. Asignamos a la variable el TextView:

```
setTextoUbicacion(findViewById(R.id.textoUbicacion));
```

- Por último, generaremos el servicio y llamaremos a los métodos getLocation y setView del servicio.
- Ahora podemos probar nuestra aplicación. Como no lo hemos programado para que el usuario de permisos de GPS a la aplicación, nos va a generar un error
- Para que el error no se produzca lo que vamos a hacer es asignar manualmente el servicio. Iremos a Setting del dispositivo y le daremos los persmisos de localización.
- Cuando lo ejecutemos nos mostrará la ubicación que tenemos en ese momento.
- Con la versión Artic Fox no funciona, en Bumblebee si.



Conexiones Bluetooth

Conexiones Bluetooth

- La plataforma de Android incluye compatibilidad con la pila de red Bluetooth, la cual permite que un dispositivo intercambie datos de manera inalámbrica con otros dispositivos Bluetooth.
- El marco de trabajo de la aplicación proporciona acceso a la funcionalidad Bluetooth mediante las API de Bluetooth de Android.
- Estas API permiten a las aplicaciones conectarse de manera inalámbrica con otros dispositivos Bluetooth y habilitan las funciones inalámbricas punto a punto y multipunto.

Conexiones Bluetooth

- Con las API de Bluetooth, una aplicación de Android puede realizar lo siguiente:
  - Buscar otros dispositivos Bluetooth
  - Consultar el adaptador local de Bluetooth en busca de dispositivos Bluetooth sincronizados
  - Conectarse con otros dispositivos mediante el descubrimiento de servicios
  - Transferir datos hacia otros dispositivos y desde estos
  - Administrar varias conexiones

Conexiones Bluetooth

- Las clases principales son:

Función	Clase
Obtener adaptador del sistema	BluetoothAdapter
Representar un dispositivo remoto	BluetoothDevice
Crear un servidor Bluetooth	BluetoothServerSocket
Conexión cliente con un dispositivo	BluetoothSocket

Conexiones Bluetooth

- Pasos:
  - Comprobar disponibilidad de Bluetooth
  - Activar/desactivar Bluetooth desde la app
  - Descubrir dispositivos cercanos (startDiscovery() + BroadcastReceiver)
  - Establecer comunicación mediante sockets RFCOMM.

Conexiones Bluetooth

- Para poder usar las funciones de Bluetooth hay que declarar tres permisos en AndroidManifest:

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

- Vamos a hacer una práctica para ver como activar el bluetooth.
- Creamos un nuevo proyecto con el nombre AplicacionBluetooth.
- Incluimos los permisos en AndroidManifest.
- Después iremos al MainActivity.java y modificaremos el método onCreate.

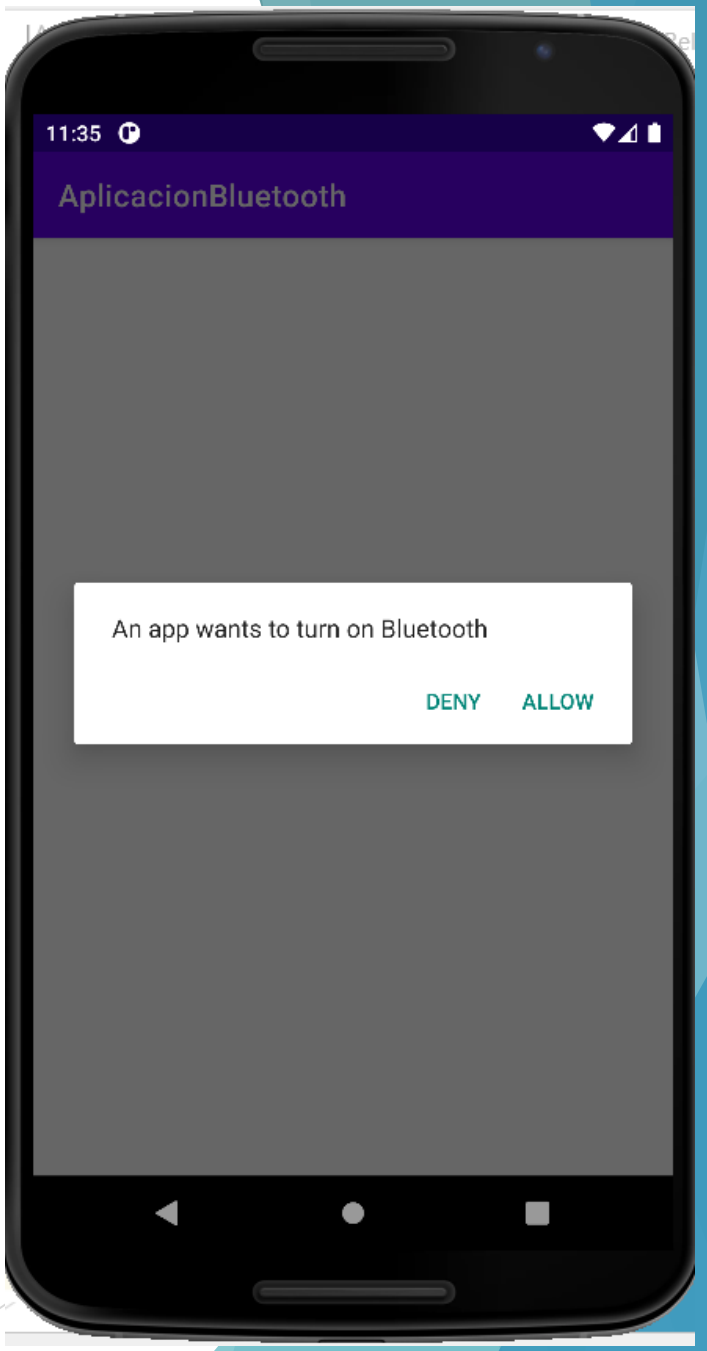
Conexiones Bluetooth

- Incluiremos unas líneas de códigos que comprobarán si nuestro dispositivo es compatible con bluetooth.
- Si no fuera compatible mostraría un mensaje en el que nos diría que no es compatible y no puede utilizarlo.
- Para poder habilitar la opción del bluetooth tendremos que añadir un condicional para comprobar si está activo o no. Si no está activo el dispositivo nos mostrará un mensaje preguntando si queremos activarlo o no.
- El código total sería:

ity_main.xml | MainActivity.java | AndroidManifest.xml

```java
import android.app.AlertDialog;
import android.bluetooth.BluetoothAdapter;
import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {
    public static int REQUEST_BLUETOOTH = 1;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        BluetoothAdapter BTAdapter = BluetoothAdapter.getDefaultAdapter();
        BTAdapter = BluetoothAdapter.getDefaultAdapter();
        // Si el telefono no soporta Bluetooth mostrará un mensaje y saldrá.
        if (BTAdapter == null) {
            new AlertDialog.Builder( context: this)
                    .setTitle("No compatible")
                    .setMessage("Tu teléfono no es compatible con Bluetooth")
                    .setPositiveButton( text: "Exit", new DialogInterface.OnClickListener() {
                        public void onClick(DialogInterface dialog, int which) {
                            System.exit( status: 0);
                        }
                    })
                    .setIcon(android.R.drawable.ic_dialog_alert)
                    .show();
```

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    BluetoothAdapter BTAdapter = BluetoothAdapter.getDefaultAdapter();
    //BTAdapter = BluetoothAdapter.getDefaultAdapter();
    if (BTAdapter==null){
        new AlertDialog.Builder( context: this)
                .setTitle("No compatible")
                .setMessage("Tu teléfono no es compatible con Bluetooth")
                .setPositiveButton( text: "Exit", new DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(DialogInterface dialog, int which) {
                        System.exit( status: 0);
                    }
                })
                .setIcon(R.drawable.ic_dialog_alert)
                .show();
    }else{
    if(!BTAdapter.isEnabled()){
        Intent enableBtIntent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
        startActivity(enableBtIntent);
    }}
}
```



File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help

EjemploBluetooth > app > src > main > java > com > example > ejemplobluetooth > MainActivity > onCreate

activity_main.xml | MainActivity.java | AndroidManifest.xml

```
13
14      @Override
15      protected void onCreate(Bundle savedIn
16          super.onCreate(savedInstanceState)
17          setContentView(R.layout.activity_m
18          BluetoothAdapter BTAdapter = Bluet
19          //BTAdapter = BluetoothAdapter.get
20          if (BTAdapter==null){
21              new AlertDialog.Builder( context
22                      .setTitle("No compatib
23                      .setMessage("Tu teléfo
24                      .setPositiveButton( tex
25                          @Override
26                          public void onClic
27                              System.exit( st
28                          }
29                      })
30                      .setIcon(R.drawable.ic
31                      .show();
32          }else{
33          if(!BTAdapter.isEnabled()){
34              Intent enableBtIntent = new In
35              startActivity(enableBtIntent);
36          }}
37      }
38  }
```

Launch succeede

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Run

Launch succeeded (moments ago)

Google Nexus 6P (1440x2560, 560dpi) - 192.168.149.101 - Genymotion

5:17

EjemploBluetooth

No compatible

Tu teléfono no es compatible con Bluetooth

EXIT

Free for personal use

Conexiones Bluetooth

- Este código lo podríamos utilizar si nuestra aplicación necesita utilizar el bluetooth para realizar alguna de sus funciones.
- Por ejemplo, si tenemos un smartwatch y queremos conectarlo con el móvil lo primero que va a comprobar es si tenemos habilitado el bluetooth. Si no lo tuviéramos habilitado nos pedirá que lo hagamos para poder mandarse datos entre los dos dispositivos.
- En caso de estar HABILITADO y TENER LOS PERMISOS NECESARIOS podríamos por ejemplos iniciar una búsqueda de dispositivos a los que conectar.

Desde desde algunas versiones recientes el emulador reporta un BluetoothAdapter "falso" (stub) aunque NO haya soporte real de Bluetooth.
Pero luego, las acciones no tienen consecuencia.

```
if (requestCode == REQUEST_BLUETOOTH) {
    if (grantResults.length > 0 &&
            grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        // Usuario aceptó
        boolean ok = BTAdapter.startDiscovery();
        Log.d( tag: "TEST",  msg: "startDiscovery: " + ok);
    } else {
        // Usuario denegó
        Toast.makeText( context: this,  text: "Permiso BLUETOOTH DENEGADO por el usuario", Toast.LENGTH_SHORT).show();
    }
}
```

Conexiones a Internet de una APP

Conexiones HTTP/HTTPS

- Diferencias entre HTTP y HTTPS → **SEGURIDAD**

- Métodos HTTP más comunes:
  - **GET:** Recupera datos del servidor.
  - **POST:** Envía datos al servidor para crear un nuevo recurso
  - **PUT:** Actualiza un recurso existente en el servidor.
  - **DELETE:** Elimina un recurso del servidor.

Conexiones HTTP/HTTPS

- Cabeceras (Headers), parámetros y cuerpos de mensaje:
  - Piezas adicionales de información que se envían junto con una solicitud o respuesta HTTP
  - Proporcionan metadatos importantes sobre la solicitud o respuesta y ayudan tanto al cliente como al servidor a entender cómo manejar los datos
  - Las cabeceras pueden incluir información como el tipo de contenido, directivas de control de caché, cookies, etc.

- Estado y códigos de respuesta HTTP (200, 404, 500…):
  - 200 OK: La solicitud se ha realizado correctamente.
  - 3xx Redirección: El servidor redirecciona a otra URL
  - 404 No encontrado: No se ha encontrado el recurso solicitado.
  - 500 Error interno del servidor: Se ha producido un error inesperado en el servidor.

Conexiones HTTP/HTTPS

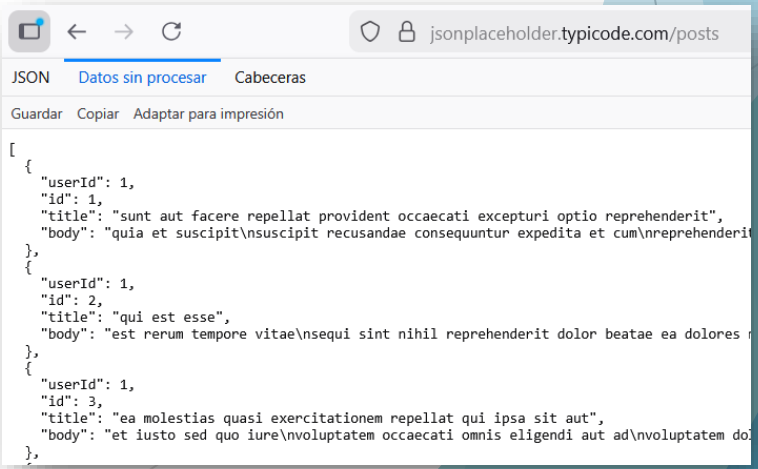
Clases	Uso
java.net.URL	Construcción de URLs
java.net.HttpURLConnection	Establecer la conexión. Configurar método de petición (setRequestMethod). Configurar cabeceras (setRequestProperty). Leer la respuesta (getInputStream() / getErrorStream()). Envío de datos (POST) mediante OutputStream.
java.net.HttpsURLConnection	Diferencias frente a HttpURLConnection. Validación de certificados SSL. Configuración de SSLSocketFactory para conexiones seguras
InputStreamReader / BufferedReader	Lectura de la respuesta
OutputStreamWriter	Envío de datos
IOException / MalformedURLException / SSLException	Gestión de excepciones

Conexiones HTTP/HTTPS

- Vamos a realizar un ejemplo en un nuevo proyecto en el que:
  - Generaremos un Fragment (HomeFragment) encargado de extraer una estructura en formato JSON de una URL (ver siguiente diapositiva), mediante una conexión HTTP.
  - La descarga la realizará en un hilo diferenciado del principal, para no bloquear la interfaz ya que la descarga puede ser lenta, pero CUIDADO al actualizar la UI porque sólo se puede hacer desde el hilo principal.
  - Los datos JSON recuperados con un GET se transformarán en objetos (Post) que se mostrarán en un RecyclerView del HomeFragment, mediante un adaptador que construiremos a medida (PostAdapter).
  - Necesitaremos varios layouts, incluyendo un fragment_home.xml con el botón y el RecyclerView y un item_post.xml que definirá la estructura del contenido de cada ítem del RecyclerView (en línea con los datos contenidos en la clase Post).

Conexiones HTTP/HTTPS

- El **formato JSON** está **basado en la sintaxis de objetos de JavaScript**, pero **utiliza una estructura de texto plano** que permite representar información de forma organizada y estructurada (**mucho más ligero que XML**)
- Un **archivo JSON** contiene **datos en pares clave-valor**, similar a cómo funcionan los objetos en JavaScript o los diccionarios en otros lenguajes. La **estructura básica de JSON** utiliza diferentes tipos de datos: **strings** (cadenas de texto), **números**, **booleanos (true/false)**, **arrays** (listas de valores), **objetos** (conjuntos de pares clave-valor) y **valores nulos.**
- Estos elementos **se combinan para crear estructuras complejas de datos** que pueden representar desde información simple hasta bases de datos completas.
- EJEMPLO



Conexiones HTTP/HTTPS

- Como continuación del ejemplo en el mismo proyecto:
  - Añadimos una llamada POST en la que enviaremos un nuevo objeto JSON al servidor.
  - Si el servidor lo procesa correctamente nos debería notificar con un 201 y devolvernos el objeto JSON que podremos mostrar en nuestro RecyclerView.
  - Necesitaremos crear un nuevo botón que lance la ejecución del envío POST.
  - NOTA: El backend de JSONPlaceholder que estamos usando en es **solo de prueba** ("fake online REST API") y por eso el servidor:
    - Recibe el JSON
    - Lo ignora
    - Nos devuelve un objeto simulado con id: 101
    - No lo guarda en ningún sitio

  Por lo tanto, no podemos ver reflejado en el servidor el cambio, aunque refresquemos el navegador apuntando a la URL utilizada: https://jsonplaceholder.typicode.com/posts